数据结构树与 leetcode 练习

在 leetcode 里面用 C 语言刷数据结构的题目时，会遇到许多 Python，Java 没有的问题，如内存开销要手动完成，还得额外注意参数传递，同时 C 语言没有它们现成的库可以使用，因此实现难度较大。

线性表部分较简单，但是到树的部分，真的需要好好进行总结，终点解决两部分：一是算法，二是 C 语言函数的书写。

C 语言函数书写包括三点：函数的类型，参数表，和参数传递。

一、树的遍历

1. 二叉树的前序，中序，后序遍历

实现前序，中序，后序遍历，通常写一个它们各自的函数，再在 leetcode 给的指定主函数里面调用它即可。在写各自函数的时候一定要写好参数表和参数传递。由于是递归方式书写，为做好参数的传递，存储结果的数组等等都放在形参表中即可。

前序的递归实现如下(lc-144)：

```
void preorder(struct TreeNode* root,int *result,int *returnSize)
{
    if(root==NULL) return;
    result[(*returnSize)++]=root->val;
    preorder(root->left,result,returnSize);
    preorder(root->right,result,returnSize);
}
```

中序的递归实现如下(lc-94)：

```
void inorder(struct TreeNode* root,int*result,int *returnSize)
{
    if(root==NULL) return;
    inorder(root->left,result,returnSize);
    result[(*returnSize)++]=root->val;
    inorder(root->right,result,returnSize);
}
```

后序的递归实现如下(lc-145)：

```
void postorder(struct TreeNode* root,int *result,int *returnSize)
{
    if(root==NULL) return;
    postorder(root->left,result,returnSize);
    postorder(root->right,result,returnSize);
    result[(*returnSize)++]=root->val;
}
```

最后将前面写好的三种遍历方式调用即可，主函数就是负责对函数里面的参数进行一个初始化，以中序遍历为例：

```
int *inorderTraversal(struct TreeNode* root,int *returnSize)
{
    int* result=(int *)malloc(100*sizeof(int ));
    *returnSize=0;
    inorder(root,result,returnSize);
    return result;
}
```

递归虽然写法简单，但要深刻理解遍历原理，须要用非递归书写。

先序遍历的非递归函数：

```
void preorder(struct TreeNode* root, int* result, int* returnSize)
{
    if (root == NULL) return;
    struct TreeNode** stack = malloc(sizeof(struct TreeNode*) * 1000);
    int top = -1;
    stack[++top] = root;
    while (top != -1)
    {
        struct TreeNode* node = stack[top--];
        result[(*returnSize)++] = node->val;
        if (node->right) stack[++top] = node->right;
        if (node->left)  stack[++top] = node->left;
    }
    free(stack);
}
```

先序遍历永远是**根-左-右**的顺序遍历，第一个入栈的是根，第一个出栈（读取）的也是根，然后才是找左右子树，注意先入栈的是右子树，再入栈的是左子树，由于出栈是逆序的，所以最后出栈的顺序是左子树和右子树，先序遍历较简单，使用一个栈即可完成任务。

中序遍历的非递归函数如下：

```
void inorder(struct TreeNode* root,int *result,int *returnSize)
{
    struct TreeNode** stack=malloc(1000*sizeof(struct TreeNode* ));
    int top=-1;
    struct TreeNode* curr=root;
    while(curr!=NULL || top!=-1)
    {
        while(curr!=NULL)
        {
            stack[++top]=curr;
            curr=curr->left;
        }
        curr=stack[top--];
        result[(*returnSize)++]=curr->val;
        curr=curr->right;
```

```
    }
    free(stack);
}
```

中序遍历和先序遍历类似，但是实现相对复杂，多了一个判断条件curr!=NULL,为什么会这样？

这是因为中序遍历的遍历方式是左-根-右的顺序，根结点不是第一时间访问的，所以 curr 指针就代表左子树有没有走完，一定要左子树走完，才能进行下一步，因此循环条件不只是栈不为空，还要对左子路的一个判断才行。下面是最复杂的后序遍历，需要借助两个栈，这也是后序遍历的顺序所决定的。

```
void postorder(struct TreeNode* root, int* result, int* returnSize)
{
    if (!root) return;
    struct TreeNode** s1 = malloc(sizeof(struct TreeNode*) * 1000);
    struct TreeNode** s2 = malloc(sizeof(struct TreeNode*) * 1000);
    int top1 = -1, top2 = -1;
    s1[++top1] = root;
    while (top1 != -1)
    {
        struct TreeNode* node = s1[top1--];
        s2[++top2] = node;
        if (node->left)  s1[++top1] = node->left;
        if (node->right) s1[++top1] = node->right;
    }
    while (top2 != -1)
        result[(*returnSize)++] = s2[top2--]->val;
    free(s1); free(s2);
}
```

后序遍历需要双栈实现，第一个栈是辅助栈，第二个栈才专门负责出元素来访问，**借助第一个栈把第二个栈压成从栈顶到栈底依次是：左-右-根的顺序**，最后从栈二出栈，得到后序序列。

了解了三种遍历方式的代码，很明显，递归的思路简单，栈的入栈出栈全部交给系统栈来执行，不用手动调试，但是非递归的算法能让我们更深刻的理解遍历原理，对于后面的变式题，就能有一战之力。

2. 二叉树遍历的进阶算法—Morris 遍历

Morris 遍历是一种二叉树的遍历方法，它通过巧妙地利用树中的空闲指针（即左子节点或右子节点为空时的指针）来实现**中序遍历**或**前序遍历**，从而避免了使用额外的栈或递归，达到了 O(1)的空间复杂度，同时保持 O(n)的时间复杂度。

O(1)的空间复杂度，保持 O(n)的时间复杂度。这句话如何理解？

可以这样理解：传统的遍历方式都需要借助栈，不论是递归的系统栈还是非递归的手工栈，根源是往下递之后，回来的这个动作必须需要栈才可以。

这就使得算法的空间复杂度达到了 O(n)。但是 Morris 遍历算法采用对树局部线索化的方式，避免了使用栈才能返回的条件，线索化之后可以直接找到后继元素，找到后继元素之后过河拆桥，去除线索化，不影响树的结构。

综上所述：**Morris 遍历是一个对树进行局部线索化，在局部遍历后去除线索化的一个过程**（注意线索化，遍历，去除是同时进行的）。

看到的情况	含义	动作
没有左孩子	自己就是当前最小的节点	输出自己，往右走（cur =cur->right）
有左孩子，前驱右指针为空	第一次来，左边还没逛	织票（pred->right = cur），往左走
有左孩子，前驱右指针指向自己	第二次来，左边逛完了	撕票（pred->right = NULL），输出自己，往右走

中序遍历的 Morris 优化算法实现如下(lc-94)：

```
void inorder(struct TreeNode* root,int *result,int* returnSize)
{
    struct TreeNode* curr=root;
    struct TreeNode* pred;
    *returnSize=0;
    while(curr!=NULL)
    {
        if(curr->left==NULL)
        {
            result[(*returnSize)++]=curr->val;
            curr=curr->right;
        }
        else
        {
            pred=curr->left;
            while(pred->right!=NULL && pred->right!=curr) {pred=pred->right;}
            if(pred->right==NULL)
            {
                pred->right=curr;
                curr=curr->left;
            }
            else
            {
                pred->right=NULL;
                result[(*returnSize)++]=curr->val;
                curr=curr->right;
            }
        }
    }
}
```

代码清晰的展示了这个算法的具体步骤，涵盖前面表中列出的三种情况，从 curr 出发先看有没有左孩子，没有左孩子直接找右孩子；有左孩子则一直找到左子树的最右结点即 curr 的前驱，进行一次局部线索化，然后把这一部分遍历，结束后撕票即可。Morris 算法属于是遍历的**优化**算法。

3. 二叉树的层序遍历

二叉树的层序遍历非常直观，就是一行一行去数树的元素，在算法实现的时候往往需要借助一个辅助队列，每次出元素的同时将它的孩子加入到队列中，循环下去直到所有的结点被遍历，结束。

```
int**levelOrder(struct TreeNode* root, int* returnSize, int**returnColumnSizes)
{
    if(root==NULL)
    {
        *returnSize=0;
        *returnColumnSizes=NULL;
        return NULL;
    }
    struct TreeNode* Queue[20000];
    int front=0,rear=0;
    int level=0,last=1,col=0;
    int **ans=(int **)malloc(20000*sizeof(int *));
    *returnColumnSizes=(int *)malloc(20000*sizeof(int ));
    *returnSize=0;
    Queue[++rear]=root;
    (*returnColumnSizes)[level]=1;
    ans[level]=(int *)malloc(1*sizeof(int ));
    while(front<rear)
    {
        struct TreeNode* p=Queue[++front];
        ans[level][col++]=p->val;
        if(p->left) Queue[++rear]=p->left;
        if(p->right) Queue[++rear]=p->right;
        if(front==last)
        {
            (*returnColumnSizes)[level]=col;
            level++;
            *returnSize=level;
            int nextlevelsize=rear-front;
            if(nextlevelsize>0)
            {
                ans[level]=(int*)malloc(nextlevelsize*sizeof(int ));
                col=0;
            }
            last=rear;
        }
    }
    return ans;
}
```

这是二叉树层序遍历最基础的遍历写法，参数表中的*returnSize 是二叉树最后的总层数，*returnColumnSizes 是一个数组，用于统计每一层包含结点的个数。

函数内部还设置了二维数组 ans，统计树的全部信息，第一个维度是层，层内维度是用来存储每一个结点的值，每一层的长度不是固定的，而是动态分配以节约空间，最后存储的样子就是一棵放倒的二叉树。

last 指针是用来判断何时进行换行，当 last==front 的时候换。

以下是层序遍历的精简版小程序，适合 debug：

```
void levelOrderPrint(struct TreeNode* root)
{
    struct TreeNode* q[2005];
    int front = 0, rear = 0;
    if (root != NULL) q[rear++] = root;
    while (front < rear)
    {
        int n= rear-front;
        for (int i = 0; i < n; i++)
        {
            struct TreeNode* p = q[front++];
            printf("%d ", p->val);
            if (p->left  != NULL) q[rear++] = p->left;
            if (p->right != NULL) q[rear++] = p->right;
        }
        puts("");
    }
}
```

4. 二叉树层序遍历的变式题目

4.1 二叉树的锯齿形层序遍历

问题：给你二叉树的根节点 root ，返回其节点值的锯齿形层序遍历。（即先从左往右，再从右往左进行下一层遍历，以此类推，层与层之间交替进行）。

首次做这个题，我简单认为可以维护两个缓冲结构——偶数层（从左到右读）用一个队列，奇数层（从右到左读）用一个栈。

偶数层出队一个节点就把它的左、右孩子压进栈；奇数层弹出一个节点就把它的左、右孩子入队；层号交替，结构交替。代码如下（错误代码）：

```
zigzagLevelOrder(root):
    若 root 为空 → 返回 空
    Q ← 空数组（当队列用：尾入、头出）;S ← 空数组（当栈用：尾进、尾出）
    Q 入队 root;level ← 0
    无限循环:
    ├── level 为偶数（本层期望输出 左→右）:
    │       若 Q 为空 → 跳出
    │       vals ← []
    │       S 清空
```

```
    │       重复 Q 当前元素个数 次:
    │           node ← Q 出队头                ★ 祸根：FIFO 原样吐出入队顺序
    │           vals 追加 node.val
    │           node.left  存在 → S 压栈
    │           node.right 存在 → S 压栈
    │       答案集 追加 vals
    └── level 为奇数（本层期望输出 右→左）:
            若 S 为空 → 跳出
            vals ← []
            Q 清空
            重复 S 当前元素个数 次:
                node ← S 弹栈顶
                vals 追加 node.val
                node.left  存在 → Q 入队尾
                node.right 存在 → Q 入队尾
            答案集 追加 vals
    level ← level + 1
返回 答案集
```

这个算法对于简单的树适用，但形状一复杂就挂。

例如 [1,2,3,4,null,null,5]，期望 [[1],[3,2],[4,5]]，实际输出 [[1],[3,2],[5,4]]，根因是队列是 LILO 的性质，上一行的是栈的逆序输入，而且每次读取的孩子也是逆序，队列不会改变顺序,所以继续把需要顺序输出的行逆序输出，因此队列不可用，必须使用双栈才能实现题目的要求。（一定记住，逆+逆=顺）。

双栈实现的正确代码如下(1c-103)：

```c
#define M 200000
int** zigzagLevelOrder(struct TreeNode* root, int* returnSize, int**
returnColumnSizes)
{
    if (root == NULL)
    {
        *returnSize = 0;
        *returnColumnSizes = NULL;
        return NULL;
    }
    struct TreeNode** Stack1 = (struct TreeNode**)malloc(M * sizeof(struct
TreeNode*));
    struct TreeNode** Stack2 = (struct TreeNode**)malloc(M * sizeof(struct
TreeNode*));
    int top1 = -1, top2 = -1;
    int level = 0;
    int** ans = (int**)malloc(M * sizeof(int*));
    *returnSize = 0;
    *returnColumnSizes = (int*)malloc(M * sizeof(int));
```

```
    Stack2[++top2] = root;
    while (1)
    {
        if (level % 2 == 0)
        {
            if (top2 == -1) break;
            int levelSize = top2 + 1;
            int* level_vals = (int*)malloc(levelSize * sizeof(int));
            (*returnColumnSizes)[*returnSize] = levelSize;
            top1 = -1;
            for (int i = 0; i < levelSize; i++)
            {
                struct TreeNode* node = Stack2[top2--];
                level_vals[i] = node->val;
                if (node->left)  Stack1[++top1] = node->left;
                if (node->right) Stack1[++top1] = node->right;
            }
            ans[(*returnSize)++] = level_vals;
        }
        else
        {
            if (top1 == -1) break;
            int levelSize = top1 + 1;
            int* level_vals = (int*)malloc(levelSize * sizeof(int));
            (*returnColumnSizes)[*returnSize] = levelSize;
            top2 = -1;
            for (int j = 0; j < levelSize; j++)
            {
                struct TreeNode* node = Stack1[top1--];
                level_vals[j] = node->val;
                if (node->right) Stack2[++top2] = node->right;
                if (node->left)  Stack2[++top2] = node->left;
            }
            ans[(*returnSize)++] = level_vals;
        }
        level++;
    }
    free(Stack1);
    free(Stack2);
    return ans;
}
```

上面的代码是遍历的高配版，不过它能够更好的体现双栈实现的具体细节，其实就是给普通层序遍历增设了两个缓冲数据结构。

最初的“队列 + 栈交替”并非全错：偶数层→奇数层那一半完全正确；错的是奇数层→偶数层那一半——它把 FIFO 队列当成了会反转的盒子。修复后两个结构都是栈，这就是 LeetCode 103 的标准双栈解法，不再存在“有些用例过不了”的结构性缺陷。

手动计算几个例子，对比两种方法：

输入	期望输出	原版实际	双栈版
[1,2,3,4,null,null,5]	[[1],[3,2],[4,5]]	[[1],[3,2],[5,4]] ✗	✓
[3,9,20,null,null,15,7]	[[3],[20,9],[15,7]]	✓	✓
[1,2,3,4,5,6,7]	[[1],[3,2],[4,5,6,7]]	[[1],[3,2],[6,7,4,5]] ✗	✓
[1,null,2,null,null,null,3]	[[1],[2],[3]]	✓	✓
[1]	[[1]]	✓	✓
空树	[]	✓	✓

最后进行批量测试，对比两个方法：

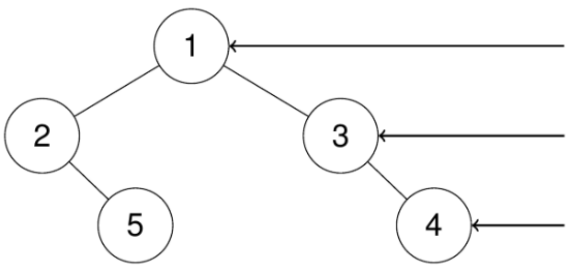
验证项	结果
3000 棵随机树：原版(队列+栈交替) 错误数	776 棵 ≈ 26%
3000 棵随机树：双栈版 错误数	0 棵
右斜深链 1500 层	双栈版 PASS
满二叉树 12 层（4095 节点，最宽层 2048）	双栈版 PASS

4.2 从下向上层序遍历二叉树

从下向上遍历二叉树，无非就是将顺序层序遍历的遍历结果数组和 ans 逆置，考察顺序表的就地逆置思想，很简单，在最开始写好的普通层序遍历的基础上，函数的 return ans 之前补上对 ans 和 returnColumnSizes 的逆置 for 循环即可。代码如下(lc-107)：

```
    for(int i=0;i<*returnSize/2;i++)
    {
        int* temp=ans[i];
        ans[i]=ans[*returnSize-i-1];
        ans[*returnSize-i-1]=temp;
    }
    for(int j=0;j<*returnSize/2;j++)
    {
        int temp=(*returnColumnSizes)[j];
        (*returnColumnSizes)[j]=(*returnColumnSizes)[*returnSize-j-1];
        (*returnColumnSizes)[*returnSize-j-1]=temp;
    }
```

4.3 二叉树的右视图



给定一个二叉树的 根节点 root，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

这个题的本质是对树层序遍历，找出每一层的最后一个结点，返回一个数组。

具体代码实现如下(lc-199)：

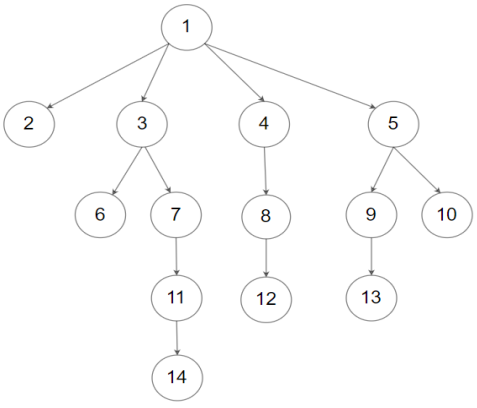
```
int* rightSideView(struct TreeNode* root, int* returnSize)
{
    int *result=(int *)malloc(100*sizeof(int ));
    struct TreeNode* Queue[1000];
    int front=0,rear=0;
    int last=1;
    *returnSize=0;
    Queue[++rear]=root;
    if(root==NULL) return NULL;
    while(front<rear)
    {
        struct TreeNode* node=Queue[++front];
        if(node->left) Queue[++rear]=node->left;
        if(node->right) Queue[++rear]=node->right;
        if(front==last)
        {
            result[(*returnSize)++]=node->val;
            last=rear;
        }
    }
    return result;
}
```

5. N 叉树的遍历

N 叉树的遍历，其实是二叉树遍历的推广，最大的不同体现在两点：一是结构体的定义不同，结构体不再是 left 和 right，而是换成了孩子结点的数组和长度；二是遍历的方式不同，使用 for 循环对结点遍历。

最值得主义的一点是，N 叉树的遍历有先序遍历，后序遍历，以及层序遍历，但就是没有中序遍历，因为中序这个词在 N 叉树是没有定义的，只是二叉树的专有。

下面书写 N 叉树的遍历算法，举个例子：



5.1 先序遍历(lc-589)

递归算法如下：

```
void dfs(struct Node* root,int *result,int * returnSize)
{
    if(root==NULL) return;
    result[(*returnSize)++]=root->val;
    for(int i=0;i<root->numChildren;i++)
    {
        dfs(root->children[i],result,returnSize);
    }
}
```

先写好辅助函数 dfs，最后结果返回到 result 数组里面，returnSize 是层数。

非递归算法依旧是借助一个手动栈实现，和普通先序一样，代码如下：

```
void dfs(struct Node* root, int* result, int* returnSize)
{
    if (root == NULL) return;
    struct Node** stack = malloc(sizeof(struct Node*) * 10000);
    int top = -1;
    stack[++top] = root;
    while (top != -1)
    {
        struct Node* node = stack[top--];
        result[(*returnSize)++] = node->val;
        for (int i = node->numChildren - 1; i >= 0; i--)
        {
            if (node->children[i])
                stack[++top] = node->children[i];
        }
    }
    free(stack);
}
```

5.2 后序遍历(lc-590)

递归算法如下：

```
void dfs(struct Node*root,int *result,int *returnSize)
{
    if(root==NULL) return;
    for(int i=0;i<root->numChildren;i++)
    {
        dfs(root->children[i],result,returnSize);
    }
    result[(*returnSize)++]=root->val;
}
```

先写好辅助函数 dfs，最后结果返回到 result 数组里面。
一个栈负责入元素给另一个栈，另一个栈负责将元素逆序输出得到后序序列。
非递归算法依旧是借助两个手动栈实现，和普通后序一样，代码如下：

```
void dfs(struct Node* root, int* result, int* returnSize)
{
    if (root == NULL) return;
    struct Node** s1 = malloc(sizeof(struct Node*) * 10000);
    struct Node** s2 = malloc(sizeof(struct Node*) * 10000);
    int top1 = -1, top2 = -1;
    s1[++top1] = root;
    while (top1 != -1)
    {
        struct Node* node = s1[top1--];
        s2[++top2] = node;
        for (int i = 0; i < node->numChildren; i++)
        {
            if (node->children[i])
                s1[++top1] = node->children[i];
        }
    }
    while (top2 != -1)
        result[(*returnSize)++] = s2[top2--]->val;
    free(s1); free(s2);
}
```

最后对 5.1 和 5.2 各自的 dfs 函数用 preorder 或 postorder 调用即可，调用过程中同时完成对 result 数组的开辟和*returnSize 的初始化。

5.3 层序遍历(lc-429)

N 叉树的层序遍历和二叉树的普通层序遍历的壳子完全一样，只是将二叉树判断左右子树的两个 if 直接换成了一个 for 循环来入队，只展示更换的部分。

具体框架如下：

```
levelOrder(root):
    若 root 为空 → 返回 空
    Q ← 空数组（从 1 号位开始编号）
    front ← 0;  rear ← 0
    last ← 1
    level ← 0;  col ← 0
    Q[++rear] ← root
    ans[0] ← 开 1 格
    当 rear > front:
    |   node ← Q[++front]
    |   ans[level][col++] ← node.val
    |   ★ 核心：孩子全部入队
    |       for (i = 0; i < node.numChildren; i++)
```

```
        |           Q[++rear] = node.children[i]
        |   若 front == last:
        |       (*returnColumnSizes)[level] ← col
        |       level ← level + 1
        |       *returnSize ← level
        |       next ← rear - front
        |       若 next > 0:
        |           ans[level] ← 开 next 格的新数组
        |           col ← 0
        |       last ← rear
        返回 ans
```

三个遍历的结果分别是：

前序 [1, 2, 3, 6, 7, 11, 14, 4, 8, 12, 5, 9, 13, 10]；

后序 [2, 6, 14, 11, 7, 3, 12, 8, 4, 13, 9, 10, 5, 1]；

层序 [[1], [2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13], [14]]

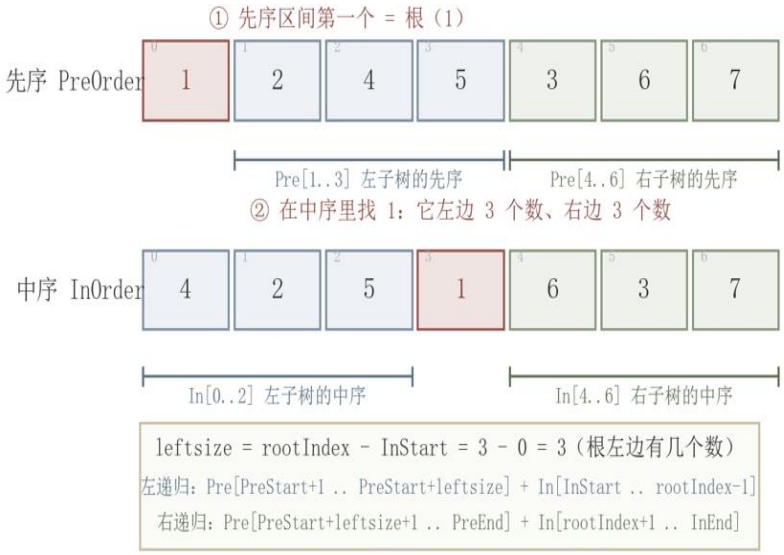
6. 遍历序列的性质与应用

6.1 前序 + 中序还原二叉树(lc-105)

```
struct TreeNode* BuildTreeFunction(int *PreOrder,int PreStart,int PreEnd,int
*InOrder,int InStart,int InEnd)
{
    if(PreStart>PreEnd || InStart>InEnd) return NULL;

    int rootval=PreOrder[PreStart];
    struct TreeNode *root=(struct TreeNode *)malloc(sizeof(struct TreeNode ));
    root->val=rootval;
    root->left=NULL;
    root->right=NULL;
    int rootIndex=InStart;
    while(rootIndex<=InEnd && InOrder[rootIndex]!=rootval)
    {
        rootIndex++;
    }
    int leftsize=rootIndex-InStart;
root->left=BuildTreeFunction(PreOrder,PreStart+1,PreStart+leftsize,InOrder,InSt
art,rootIndex-1);
root->right=BuildTreeFunction(PreOrder,PreStart+1+leftsize,PreEnd,InOrder,rootI
ndex+1,InEnd);
    return root;
}
```

流程示意图为：



先用先序序列确定出来根的值，再根据这个值扫描中序序列找到根的索引，将中序序列分为两部分，左右子树就能确定下来，最后对左右子树递归构建即可完成。

6.2 后序 + 中序还原二叉树(lc-106)

```
struct TreeNode* BuildTreeFunction(int *InOrder,int InStart,int InEnd,int
*PostOrder,int PostStart,int PostEnd)
{
    if(PostStart>PostEnd || InStart>InEnd) return NULL;
    int rootval=PostOrder[PostEnd];
    struct TreeNode *root=(struct TreeNode *)malloc(sizeof(struct TreeNode ));
    root->val=rootval;
    root->left=NULL;
    root->right=NULL;
    int rootIndex=InStart;
    while(rootIndex<=InEnd && InOrder[rootIndex]!=rootval)
    {
        rootIndex++;
    }
    int leftsize=rootIndex-InStart;
    root->left=BuildTreeFunction(InOrder,InStart,rootIndex-
1,PostOrder,PostStart,PostStart+leftsize-1);
    root->right=BuildTreeFunction(InOrder,rootIndex+1,InEnd,PostOrder,PostStart
+leftsize,PostEnd-1);
    return root;
}
```

后序 + 中序还原二叉树的示意图和前序 + 中序还原二叉树的示意图类似
都是先找到根节点；
再对左右子树递归，构建出来二叉树。
最终，对这两个函数分别调用即可
以第一个函数为例：

```
struct TreeNode* buildTree(int* preorder, int preorderSize, int* inorder, int
inorderSize)
{
    if(preorder==NULL || inorder==NULL || preorderSize==0 ||inorderSize==0)
    {
        return NULL;
    }
    return BuildTreeFunction(preorder,0,preorderSize-1,inorder,0,inorderSize-
1);
}
```

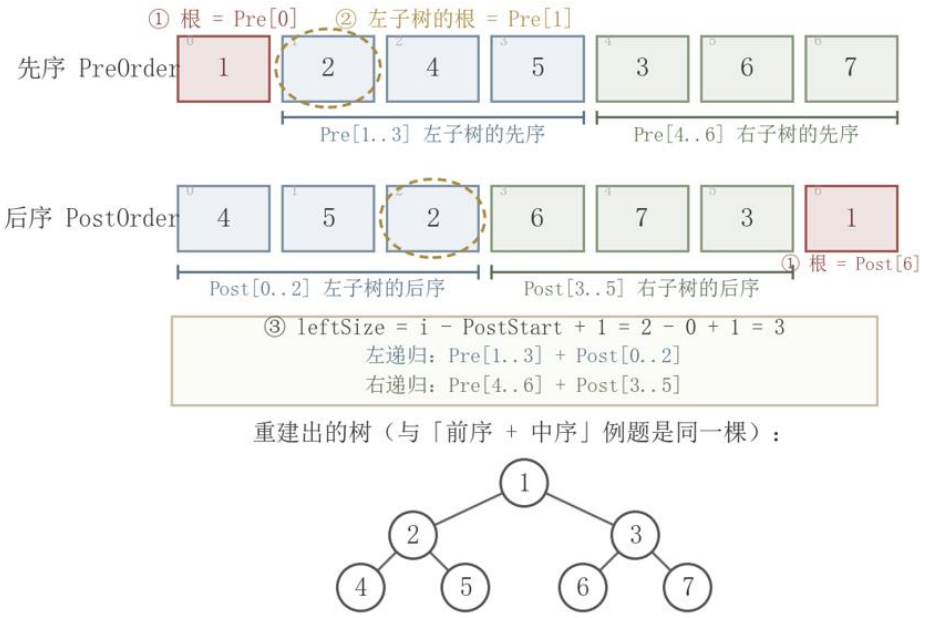
6.3 利用前序和后序构造二叉树

```
struct TreeNode* build(int *preorder,int prestart,int preend,int *postorder,int
poststart,int postend)
{
    if(prestart>preend) return NULL;
    struct TreeNode* root=(struct TreeNode*)malloc(sizeof(struct TreeNode ));
    root->val=preorder[prestart];
    root->left=root->right=NULL;
    if(prestart==preend) return root;
    int leftrootval=preorder[prestart+1];
    int i=poststart;
    while(postorder[i]!=leftrootval) i++;
    int leftsize=i-poststart+1;
    root->left=build(preorder,prestart+1,prestart+leftsize,postorder,poststart,
i);
    root->right=build(preorder,prestart+1+leftsize,preend,postorder,i+1,postend
-1);
    return root;
}
```

测试函数如下：

```
struct TreeNode* constructFromPrePost(int* preorder, int preorderSize, int*
postorder, int postorderSize)
{
    if(preorder==NULL||postorder==NULL||preorderSize==0||postorderSize==0)
         return NULL;
    return build(preorder,0,preorderSize-1,postorder,0,postorderSize-1);
}
```

流程示意图如下：



重建出的树（与「前序 + 中序」例题是同一棵）：

前序定根，而前序里紧跟在根后面的那个元素，就是左子树的根——拿它到后序里定位，一刀切出左子树的大小。

二、树的搜索

1. 深度优先搜索（DFS）

1.1 路径类问题（根到叶子）

lc-257 二叉树的所有路径：给你一个二叉树的根节点 root。按任意顺序，返回所有从根节点到叶子节点的路径。叶子节点是指没有子节点的节点。

```
void dfs(struct TreeNode* node, char* path, int len, char** res, int*
returnSize)
{
    if (node == NULL) return;
    if (len == 0) {len += sprintf(path + len, "%d", node->val);}
    else {len += sprintf(path + len, "->%d", node->val);}
    if (node->left == NULL && node->right == NULL)
    {
        char* onePath = (char*)malloc(sizeof(char) * (len + 1));
        strcpy(onePath, path);
        res[*returnSize] = onePath;
        (*returnSize)++;
        return;
    }
    dfs(node->left, path, len, res, returnSize);
    dfs(node->right, path, len, res, returnSize);
}
```

根据题目要求，结果是类似["1->2->5","1->3"]这样的输出，用 C 语言实现起来会稍微复杂一些，最后的输出存在 res 这样的二维字符型数组里面，中间使用了 sprintf 函数，**基本用法是 len=sprintf(Path+len,%d,value)，Path+len 是数组 Path 的第 len 个位置处添加元素，中间的是格式，value 是添加的元素值，最后返回新数组的长度更新 len**，所以函数里面先是往下 dfs，直到叶子结点后，再将这个路径放入 res 矩阵中，Path 是公共缓冲区，onePath 是搬运工。

主函数调用验证结果：

```
char** binaryTreePaths(struct TreeNode* root, int* returnSize)
{
    char** res = (char**)malloc(sizeof(char*) * 1000);
    *returnSize = 0;
    if (root == NULL) return res;
    char path[1000];
    dfs(root, path, 0, res, returnSize);
    return res;
}
```

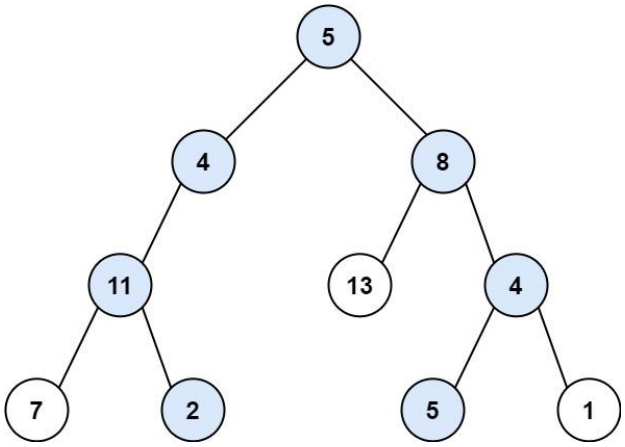
lc-112 **路径总和**：给你二叉树的根节点 root 和一个表示目标和的整数 targetSum。判断该树中是否存在根节点到叶子节点的路径，这条路径上所有节点值相加等于目标和 targetSum。如果存在，返回 true ；否则，返回 false。

```
bool hasPathSum(struct TreeNode* root, int targetSum)
{
    if(root==NULL) return false;
    targetSum=targetSum-root->val;
    if(root->left==NULL && root->right==NULL){return targetSum==0;}
    return hasPathSum(root->left,targetSum)||hasPathSum(root->right,targetSum);
}
```

使用逆向思维，直接对每一条路用 dfs，一直减权值，看最后到叶子结点上的时候 targetSum 是否为 0 即可，说实话，第一次做这个题很难想到。

lc-113 **路径总和 II**：给你二叉树的根节点 root 和一个整数目标和 targetSum，找出所有从根节点到叶子节点路径总和等于给定目标和的路径。

113 题继续沿用 112 题的思想，对 targetSum 做减法，看看叶子结点是否为 0，但是这个题需要记录符合要求的路径，实现起来还需借助一个栈，这个栈就是 path 数组，采用系统栈+手动栈的组合方法，系统栈负责扫描，手动栈负责记路线。



输入： root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
输出： [[5,4,11,2],[5,8,4,5]]

```
void dfs(struct TreeNode* root,int targetSum,int *path,int *len,int **res,int
*returnSize,int **returnColumnSizes)
{
    if(root==NULL) return;
    path[*len]=root->val;
    (*len)++;
    targetSum=targetSum-root->val;
    if(root->left==NULL && root->right==NULL)
    {
        if(targetSum==0)
        {
            int *onepath=(int*)malloc((*len)*sizeof(int ));
            for(int i=0;i<(*len);i++){onepath[i]=path[i];}
```

```
            res[(*returnSize)]=onepath;
            (*returnColumnSizes)[*returnSize]=*len;
            (*returnSize)++;
        }
    }
    else
    {
        dfs(root->left,targetSum,path,len,res,returnSize,returnColumnSizes);
        dfs(root->right,targetSum,path,len,res,returnSize,returnColumnSizes);
    }
    (*len)--;
}
```

这个 dfs 算法开头设置了 path 栈用于存路径，else 里面的两个 dfs 是系统栈在递归，len 就是 path 栈的栈顶指针，这个算法的重点就是要手动 push 和 pop，最后的 len—也不可以忘记，这是对不符合要求路线的去除，否则栈只会越来越大，得到错误结果。

还有一点值得注意，为什么每一次递归不用管 targetSum？是因为 targetSum 是值传递，而值传递传递的是原件的副本，递归一次改的是复印件，退层之后再次递归不影响 targetSum 的值。len 是指针传递，是共享的，每一次必须手动复原。

最后，为什么这里记路径要一个 onepath 中间量，原因是 path 是一个公共空间，如果直接把 path 存到 res 里面，path 在后序的递归里面还会变化，最后所有的答案会变成同一个，失去了函数的作用。

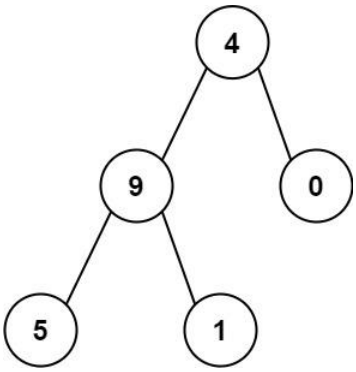
最后调用主函数：

```
int** pathSum(struct TreeNode* root, int targetSum, int* returnSize, int**
returnColumnSizes) {
    int** res = (int**)malloc(sizeof(int*) * 1000);
    *returnColumnSizes = (int*)malloc(sizeof(int) * 1000);
    *returnSize = 0;
    if (root == NULL) return res;
    int* path = (int*)malloc(sizeof(int) * 1000);
    int len = 0;
    dfs(root, targetSum, path, &len, res, returnSize, returnColumnSizes);
    free(path);
    return res;
}
```

搜索类问题和遍历类问题不一样，遍历是，单纯的系统栈或者单纯的手动栈递归即可，搜索是要记载路线，往往是系统栈干活，手动栈记录这种组合，搜索函数的写法对指针，内存管理有更高要求。

lc-129 求根节点到叶子结点数字之和：给你一个二叉树的根节点 root，树中每个节点都存放有一个 0 到 9 之间的数字。每条从根节点到叶节点的路径都代表一个数字：例如，从根节点到叶节点的路径 1 -> 2 -> 3 表示数字 123 。

计算从根节点到叶节点生成的 所有数字之和。



输入： root = [4,9,0,5,1]

输出： 1026

从根到叶子节点路径 4->9->5 代表数字 495

从根到叶子节点路径 4->9->1 代表数字 491

从根到叶子节点路径 4->0 代表数字 40

因此，数字总和 = 495 + 491 + 40 = 1026

两个思路：一是采用 257 题的 dfs 找出所有路径使用字符串形式存储起来，这个方法稳健，但是最后需要写一个 for 循环对 res 数组里面的每一个字符串使用 atoi 关键字还原为数字求和，较为麻烦；二是这个题是十进制求和的问题，直接用 **curr=surr*10+root->val 边 dfs 边求和，更快，更轻便**，这里都展示出来。

方法一，照搬 257 的 dfs，主函数调用格式为：

```
int sumNumbers(struct TreeNode* root)
{
    int returnSize=0, len=0;
    char *path=(char*)malloc(1000*sizeof(char));
    char **res=(char **)malloc(1000*sizeof(char*));
    int num,sum=0;
    if(root==NULL) return 0;
    dfs(root,path,res,&returnSize,len);
    for(int i=0;i<(returnSize);i++)
    {
        num=atoi(res[i]);
        sum+=num;
    }
    return sum;
}
```

注意标红的两行，由于函数的形参表没有给 int*returnSize，所以需要自己建立，直接定义为 int 型，调用的时候加上&即可，**不可以直接 int*returnSize=0，会报错！**

方法二，边走边求和：

```
int dfs(struct TreeNode*root,int curr)
    if(root==NULL) return 0;
    curr=curr*10+root->val;
    if(root->left==NULL && root->right==NULL) return curr;
    return dfs(root->left,curr)+dfs(root->right,curr);
```

最后提一嘴，lc-1022 **题与此题高度相似**，只是求的是二进制数之和，只用把*10 换成*2 即可得到结果，一分钟即可写完。

1.2 树的结构判断（递归比较）

lc-100 **相同的树**：给你两棵二叉树的根节点 p 和 q ，编写一个函数来检验这两棵树是否相同。如果两个树在结构上相同，并且节点具有相同的值，则认为它们是相同的。

算法很简单，如下：

```
bool isSameTree(struct TreeNode* p, struct TreeNode* q)
{
    if(p==NULL&&q==NULL) return true;
    else if(p==NULL || q==NULL) return false;
    else if(p->val!=q->val) return false;
    else return isSameTree(p->left,q->left) && isSameTree(p->right,q->right);
}
```

简单的分支+递归函数，基础题。

lc-101 **对称二叉树**：给你一个二叉树的根节点 root，检查它是否轴对称。

```
bool check(struct TreeNode *p,struct TreeNode *q)
{
    if(p==NULL && q==NULL) return true;
    else if(p==NULL || q==NULL) return false;
    else if(p->val!=q->val) return false;
    else {return check(p->left,q->right) && check(p->right,q->left);}
}
bool isSymmetric(struct TreeNode* root) {
    return check(root->left,root->right);
}
```

需要借助一个辅助函数 check，check 和前面判断两棵树相同的算法类似。

lc-572 **另一棵树的子树**：给你两棵二叉树 root 和 subRoot。检验 root 中是否包含和 subRoot 具有相同结构和节点值的子树。如果存在，返回 true；否则，返回 false 。同时二叉树 tree 的一棵子树包括 tree 的某个节点和这个节点的所有后代节点。tree 也可以看做它自身的一棵子树。

```
bool isSubtree(struct TreeNode* root, struct TreeNode* subRoot)
{
    if(root==NULL) return false;
    if (isSameTree(root, subRoot)) return true;
    return isSubtree(root->left, subRoot) || isSubtree(root->right, subRoot);
}
```

这个题外层对 root 递归，内层就是不断使用 100 题的 isSameTree 函数递归。以每一个结点为根的树和给定的子树对比，看是否一样。

1.3 树的深度/高度计算

树的深度计算基本都使用 dfs 来求解，少数用 bfs 算法，解决以下例题：

lc-104 **二叉树的最大深度**：给定一个二叉树 root，返回其最大深度。二叉树的最大深度是指从根节点到最远叶子节点的最长路径上的节点数。

```
int maxDepth(struct TreeNode* root)
{
    if(root==NULL) return 0;
    if(root->left==NULL&&root->right==NULL) return 1;
    return max(maxDepth(root->left),maxDepth(root->right))+1;
}
```

此算法较简单，无脑写递归就可以，最后的高度不要忘记加一，根结点也算。

lc-111 **二叉树的最小深度**：给定一个二叉树，找出其最小深度，最小深度是从根节点到最近叶子节点的最短路径上的节点数量。

```
int minDepth(struct TreeNode* root)
{
    if(root==NULL) return 0;
    if(root->left==NULL&&root->right==NULL) return 1;
    if(!root->left) return 1+minDepth(root->right);
    if(!root->right) return 1+minDepth(root->left);
    return 1+min(minDepth(root->left),minDepth(root->right));
}
```

找一棵树的最小深度的时候，一定要注意对左右子树是否为空的讨论，对于 max 函数，求出来的结果会越来越大，**但是 min 有可能求出来 0**，这就是左/右子树是空的情况，这实际上是错的，此时树的高度就只能是右/左子树的高度了。

lc-110 **判断平衡二叉树**：给定一棵树，判断是否是平衡二叉树

```
int height(struct TreeNode* root)
{
    if(root==NULL) return 0;
    int h1=height(root->left);
    int h2=height(root->right);
    return (h1>h2?h1:h2)+1;
}
bool isBalanced(struct TreeNode* root)
{
    if(root==NULL) return true;
    int h1=height(root->left);
    int h2=height(root->right);
    if(abs(h1-h2)>1) return false;
    return isBalanced(root->left) && isBalanced(root->right);
}
```

写一个求深度的函数，之后在主函数调用，主函数也是递归的，刚好符合平衡二叉树递归的性质，对于每一个结点的左右子树都要判断树高度差不大于 1。

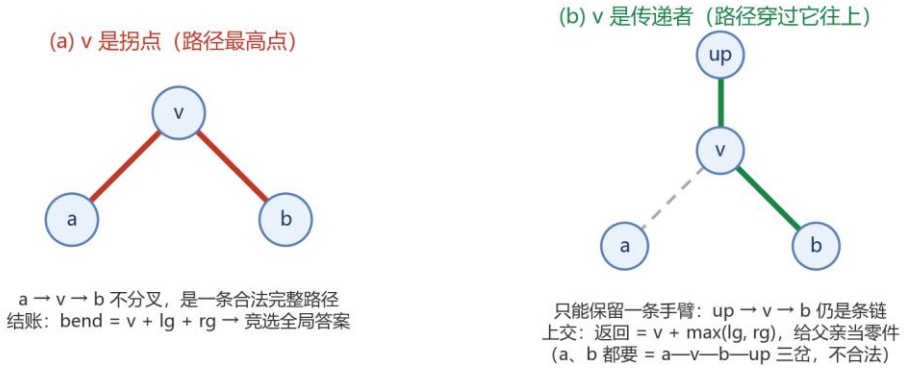
1.4 树形 DP / 后序信息传递（动态规划问题）

树结构的动态规划问题往往采用后序 dfs 的方式，动态规划问题一直是难点。

lc-124 二叉树中最大路径和：二叉树中的路径被定义为一条节点序列，序列中每对相邻节点之间都存在一条边。同一个节点在一条路径序列中 至多出现一次，该路径至少包含一个节点，且不一定经过根节点。

题目：给你一个二叉树的根节点 root ，返回其最大路径和。

这个问题为什么采用**动态规划+后序遍历**？如果单纯从上到下找，就会退化为贪心策略，很有可能在后序结点上付出更大的代价（巨大的负值），因此要先把孩子看一遍，再往上传递给父亲，这时分为了两种情况：



当树结构是第一种时直接竞选；第二种则需要向上传递，而且传递的是左右子树较优秀的那一棵，即 max(left,right)，之后会在代码里体现。

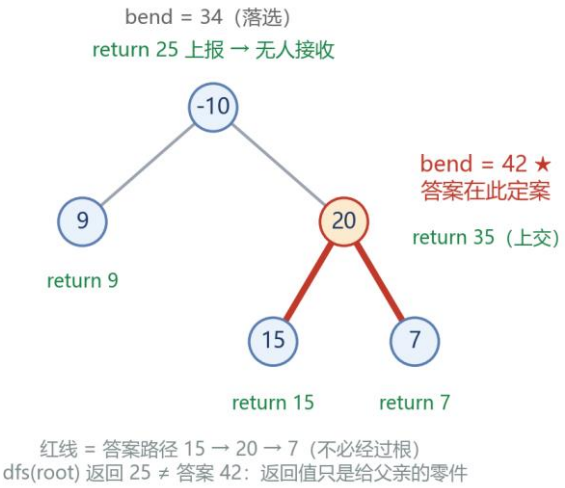
```
int maxSum=INT_MIN;
int postdfs(struct TreeNode*root)
{
    if(root==NULL) return 0;
    int left=max(postdfs(root->left),0);
    int right=max(postdfs(root->right),0);
    maxSum=max(maxSum,root->val+right+left);
    return root->val+max(left,right);
}
int maxPathSum(struct TreeNode* root)
{
    maxSum=INT_MIN;
    postdfs(root);
    return maxSum;
}
```

代码中标红的两行，第一行对应第一种情况，第二行对应第二种情况。

代码严格遵照后序 dfs 的方式，先 left 和 right，最后算上 root。

其实这个 postdfs 函数就是动态规划的转移函数。（四步法）

下面以 leetcode124 题自带的例子，标注出每一步：

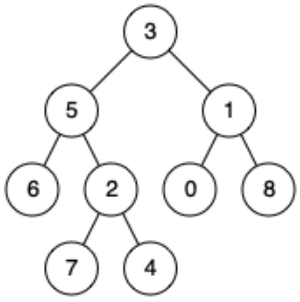


这里是为了讲 dfs 遍历的用法，引入了个 DP 题目，之后会有专门的模块讲解。

1.5 寻找祖先结点

lc-236 二叉树的最近公共祖先：给定一个二叉树，找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为：“对于有根树 T 的两个节点 p、q，最近公共祖先表示为一个节点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”例如：



输入：root = [3,5,1,6,2,0,8,null,null,7,4]，p = 5，q = 4
输出：5

```
struct TreeNode* lowestCommonAncestor(struct TreeNode* root, struct TreeNode*
p, struct TreeNode* q)
{
    if(root==NULL||p==root||q==root) return root;
    struct TreeNode* left=lowestCommonAncestor(root->left,p,q);
    struct TreeNode* right=lowestCommonAncestor(root->right,p,q);
    if(left && right) return root;
    return left?left:right;
}
```

这是寻找最近祖先的代码，严格按照后序 dfs 书写。此代码有一个细节，就是代码标红的那一行，这一行的重要作用就是剪枝，如果不剪枝，再数据量大的情况下先后序再一个个判断会非常耗时，因此，剪枝是有必要的。我亲自跑过剪枝版本和不剪枝版本的代码，二者耗时又很大差异，剪枝版明显更快。

lc-235 二叉搜索树的最近公共祖先：给定一个二叉搜索树，找到该树中两个指定节点的最近公共祖先。

这个题的代码完全可以使用 lc-236 的代码，因为 236 的树是无序的，无需的树都适用，那对于有序的 BST 更不在话下，可以直接照抄，还有另一种办法是，适当借助 BST 的性质来查找，代码如下：

```
struct TreeNode* lowestCommonAncestor(struct TreeNode* root, struct TreeNode*
p, struct TreeNode* q)
{
    if (root == NULL) return NULL;
    if (root->val > p->val && root->val > q->val)
        return lowestCommonAncestor(root->left, p, q);
    if (root->val < p->val && root->val < q->val)
        return lowestCommonAncestor(root->right, p, q);
    return root;
}
```

借助 BST 的性质，代码会更简单，当 p，q 和根比较一大一小时，那最近的祖先就是 root 结点，若都小往左走，反之向右走即可。

2. 广度优先搜索（BFS）

2.1 层序基础与变形

广度优先搜索的基础是树的层序遍历，而树的层序遍历在前面已经提到过，且有经典例题，这里就不再赘述，详情请看第一部分的层序遍历。

2.2 层深度 / 高度

bfs 逐层推进，层数天然就是深度，所以求“最短/最早到达”类问题时比 DFS 更直观。回看 dfs 部分的 lc-111 题，**使用 bfs 会比 dfs 更好，一旦遇到第一个叶子节点，当前层数就是最小深度，不需要遍历完整棵树**。

算法修改如下：

```
int bfs(struct TreeNode* root,int *returnSize)
{
    if(root==NULL) return 0;
    struct TreeNode*Queue[10000];
    *returnSize=1;
    Queue[++rear]=root;
    while(front<rear)
    {
        struct TreeNode*node=Queue[++front];
        if(node->left==NULL&&node->right==NULL) return *returnSize;
        if(node->left) Queue[++rear]=node->left;
        if(node->right) Queue[++rear]=node->right;
        if(front==last)
        {
            (*returnSize)++;
```

```
            last=rear;
        }
    }
    return *returnSize;
}
```

写法没多大变化，只不过是遇见叶子结点提前结束罢了，对了，这里是求深度，所以把*returnSize 初始化为 1，别忘了。

对于前面用 dfs 解决的 lc-104 求最大深度题，也可以使用 bfs 来做。

2.3 层内操作与聚合

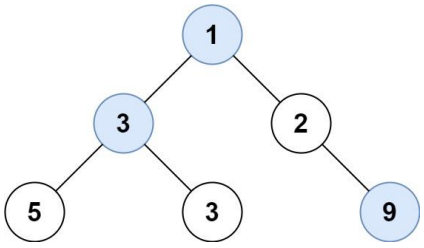
这类题的特点是：BFS 负责分层，真正的逻辑在处理“当前层”的数据。

lc-637 二叉树的层平均值：给定一个非空二叉树的根节点 root，以数组的形式返回每一层节点的平均值。

```
double *bfs(struct TreeNode* root,int *returnSize)
{
    double sum=0,aver=0;
    double *result=(double *)malloc(10000*sizeof(double ));
    struct TreeNode* Queue[10000];
    int front=0,rear=0,last=1,col=0;
    *returnSize=0;Queue[++rear]=root;
    while(front<rear)
    {
        struct TreeNode*node=Queue[++front];
        sum=sum+node->val;
        col++;
        if(node->left) Queue[++rear]=node->left;
        if(node->right) Queue[++rear]=node->right;
        if(front==last)
        {
            aver=sum/col;
            result[(*returnSize)]=aver;
            (*returnSize)++;
            last=rear;col=0;sum=0;
        }
    }
    return result;
}
double* averageOfLevels(struct TreeNode* root, int* returnSize)
{
    double *result=(double *)malloc(10000*sizeof(double ));
    *returnSize=0;
    return bfs(root,returnSize);
}
```

依旧是层序遍历的壳子，只不过多了层内的求和+求平均操作。
注意：每层结束后一定要**置空 sum 和 col 才能进入下一轮！**

lc-515 在每个树行中找最大值：给定一棵二叉树的根节点 root ，请找出该二叉树中每一层的最大值。



输入：root = [1,3,2,5,3,null,9]
输出：[1,3,9]

先设置一个全局变量 maxnum，初始化为 INT_MIN，在每一层的结尾重置。

```
int maxnum=INT_MIN;
int *bfs(struct TreeNode* root,int *returnSize)
{
    int *result=(int*)malloc(100000*sizeof(int ));
    if(root==NULL) return NULL;
    struct TreeNode* Queue[20000];
    int front=0,rear=0,last=1;
    Queue[++rear]=root;
    while(front<rear)
    {
        struct TreeNode* node=Queue[++front];
        maxnum=max(maxnum,node->val);
        if(node->left) Queue[++rear]=node->left;
        if(node->right) Queue[++rear]=node->right;
        if(front==last)
        {
            result[(*returnSize)++]=maxnum;
            maxnum=INT_MIN;
            last=rear;
        }
    }
    return result;
}
int* largestValues(struct TreeNode* root, int* returnSize)
{
    int *result=(int *)malloc(10000*sizeof(int ));
    *returnSize=0;
    return bfs(root,returnSize);
}
```

标红的部分是代码的关键，需要注意，最后主函数调用得到正确结果。

lc-116 **填充每个节点的下一个右侧节点指针**：给定一个完美二叉树，其所有叶子节点都在同一层，每个父节点都有两个子节点。填充它的每个 next 指针，让这个指针指向其下一个右侧节点。如果找不到下一个右侧节点，则将 next 指针设置为 NULL。

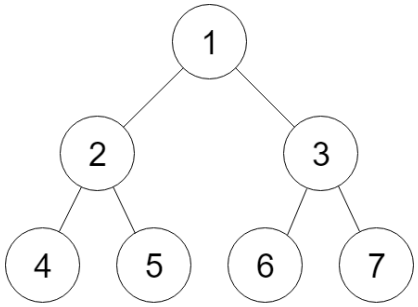


Figure A

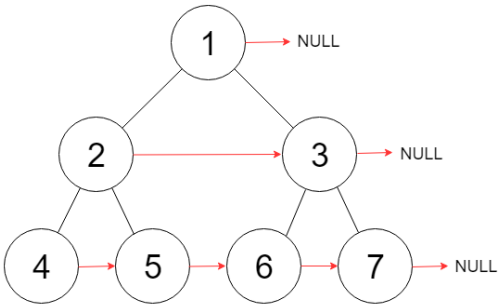


Figure B

```
struct Node* dfs(struct Node*root)
{
    if(root==NULL) return NULL;
    if(root->left)
    {
        root->left->next=root->right;
        if(root->next)
        {
            root->right->next=root->next->left;
        }
    }
    dfs(root->left);
    dfs(root->right);
    return root;
}
```

递归形式的函数，借助了完美二叉树的性质，代码实现起来较简单。

lc-117 **在 116 的基础上**，将二叉树换成不完美的，这个时候采用 bfs 更好。

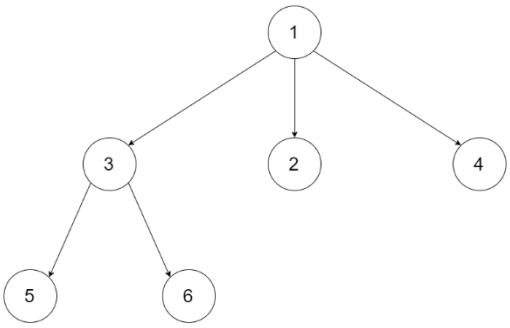
```
while(front<rear)
    {
        int length=rear-front;
        struct Node*last=NULL;
        for(int i=0;i<length;i++)
        {
            struct Node*node=Queue[++front];
            if(node->left) Queue[++rear]=node->left;
            if(node->right) Queue[++rear]=node->right;
            if(last!=NULL) last->next=node;
            last=node;
        }
    }
```

只展示 bfs 的核心部分，壳子都是一样的。

2.4 N 叉树扩展

lc-429 N 叉树的层序遍历，在前面已经提到过。

lc-559 N 叉树的最大深度：给定一个 N 叉树，找到其最大深度。




```
int maxDepth(struct Node* root)
{
    if(root==NULL) return 0;
    int maxdep=0;
    for(int i=0;i<root->numChildren;i++)
    {
        if(root->children[i]==NULL) continue;
        int dep=maxDepth(root->children[i]);
        maxdep=maxdep>dep?maxdep:dep;
    }
    return maxdep+1;
}
```

递归写在 for 循环内部，因为根有 numChildren 个孩子。

3. 二叉搜索树的搜索（BST）

3.1 BST 的查找、插入、删除

lc-700 二叉搜索树中的搜索：给定二叉搜索树（BST）的根节点 root 和一个整数值 val。你需要在 BST 中找到节点值等于 val 的节点。 返回以该节点为根的子树。如果节点不存在，则返回 null。

```
struct TreeNode* searchBST(struct TreeNode* root, int val)
{
    if(root==NULL||root->val==val) return root;
    if(val<root->val) return searchBST(root->left,val);
    else return searchBST(root->right,val);
}
```

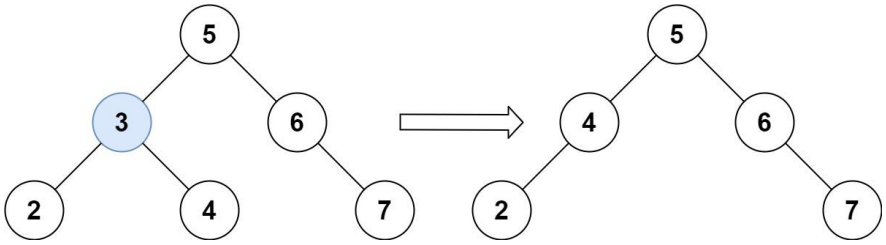
一个简单的递归即可实现，利用 BST 左小右大的性质。

lc-701 二叉搜索树的插入：给定二叉搜索树（BST）的根节点 root 和要插入树中的值 value，将值插入二叉搜索树。返回插入后二叉搜索树的根节点。输入数据保证，新值和原始二叉搜索树中的任意节点值都不同。

注意，对于一个空树，插入结点的时候要先开辟插入点的空间以及初始化再插入，后面进行递归：

```
struct TreeNode* insertIntoBST(struct TreeNode* root, int val)
{
    if(root==NULL)
    {
        struct TreeNode* node=malloc(sizeof(struct TreeNode));
        node->val=val;
        node->left=node->right=NULL;
        return node;
    }
    if(val<root->val)  root->left=insertIntoBST(root->left,val);
    else root->right=insertIntoBST(root->right,val);
    return root;
}
```

lc-450 **二叉搜索树的删除**：给定一个二叉搜索树的根节点 root 和一个值 key，删除二叉搜索树中的 key 对应的节点，并保证二叉搜索树的性质不变。返回二叉搜索树（有可能被更新）的根节点的引用。



输入：root = [5,3,6,2,4,null,7], key = 3

输出：[5,4,6,2,null,null,7]

对于二叉搜索树的删除，我们第一时间联想到普通二叉树结点的删除步骤，对于一个二叉树结点分为**三种情况**：

1）叶结点，直接 free 删除即可；

2）有一个孩子的结点，也简单，直接用孩子替换根节点即可；

3）有两个孩子的结点，需要找到其前驱或后继，用前驱或后继的值覆盖根结点的值，再删除这个前驱或后继即可。

对于 2）3）的情况最后都可以转化为 1）的情况直接 free。

我的思路是写一个中序递归函数，在函数内部实现对结点的删除，并分别讨论以上三种情况，值得注意的是，整个函数在生命周期内只会使用一次 free，而且只会删除一个结点，所以这个递归函数要十分注意参数传递的细节。

free 的三条纪律,每一条都是“每个帧只对自己负责”：

1）只 free 自己确认要删的那一个结点(命中帧)——一次成功删除,整棵调用树加起来恰好一次 free;

2）free 之前先把孩子存下来(child =root->left?root->left:root->right;),因为 free 之后不许再读 root;

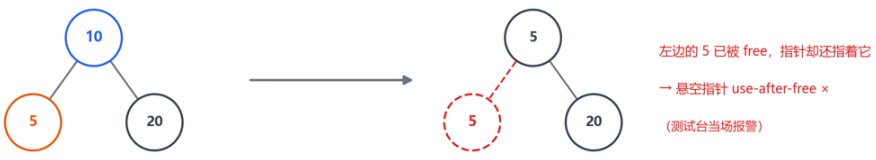
3）free 之后立刻 return 交人,绝不再碰 root。

算法实现如下：

```
struct TreeNode* findmax(struct TreeNode* root)
{
    while(root->right!=NULL) {root=root->right;}
    return root;
}
struct TreeNode* inorder(struct TreeNode* root,int key)
{
    if(root==NULL) return NULL;
    root->left=inorder(root->left,key);
    if(root->val==key)
    {
        if(root->left==NULL && root->right==NULL)
        {
            free(root);
            return NULL;
        }
        if(root->left==NULL || root->right==NULL)
        {
            struct TreeNode* child=root->left?root->left:root->right;
            free(root);
            return child;
        }
        struct TreeNode* pre=findmax(root->left);
        root->val=pre->val;
        root->left=inorder(root->left,pre->val);
        return root;
    }
    root->right=inorder(root->right,key);
    return root;
}
struct TreeNode* deleteNode(struct TreeNode* root,int key)
{
    return inorder(root,key);
}
```

findmax 函数是用来寻找树的最大值结点，最后应用在左子树里面，根据中序遍历的性质，找出来的左子树的最大值结点就是根节点的前驱结点。

函数 inorder 第三个标红的语句十分关键，代表了删除前驱结点，递归到左子树里面去直接直接执行删除叶子结点的语句就可以删除，如果没有这句话会报错。



就会像上面的例子一样，报错。

3.2 验证二叉搜索树 BST

lc-98 验证二叉搜索树：给你一个二叉树的根节点 root，判断其是否是一个有效的二叉搜索树。

```
int *inorder(struct TreeNode* root,int *result,int *len)
{
    if(root==NULL) return NULL;
    inorder(root->left,result,len);
    result[(*len)++]=root->val;
    inorder(root->right,result,len);
    return result;
}
bool isValidBST(struct TreeNode* root)
{
    int *result=(int*)malloc(10000*sizeof(int ));
    int len=0;
    inorder(root,result,&len);
    if(result==NULL || len<=1) return true;
    for(int i=0;i<len-1;i++)
    {
        if(result[i]>=result[i+1]) return false;
    }
    return true;
}
```

开一个数组存放中序遍历的序列，最后用 for 循环验证看这个数组是否递增。

3.3 第 K 小 / 范围求和

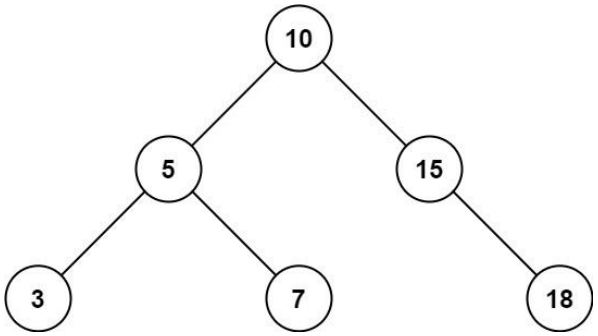
lc-230 二叉搜索树中第 K 小的元素：给定一个二叉搜索树的根节点 root，和一个整数 k，请你设计一个算法查找其中第 k 小的元素（k 从 1 开始计数）。

这个题直接在验证二叉搜索树代码的主函数里面加上：

```
    int mink=result[k-1];
    return mink;
```

lc-938 二叉搜索树的范围和：给定二叉搜索树的根结点 root，返回值位于范围[low, high]之间的所有结点的值的和。

```
int rangeSumBST(struct TreeNode* root, int low, int high)
{
    if(root==NULL) return 0;int temp=0;
    if(root->val>=low && root->val<=high) temp=temp+root->val;
    int leftsum=rangeSumBST(root->left,low,high);
    int rightsum=rangeSumBST(root->right,low,high);
    return leftsum+rightsum+temp;
}
```



输入：root = [10,5,15,3,7,null,18], low = 7, high = 15
输出：32

直接一个递归函数搞定结果。

3.4 最近的公共祖先 LCA

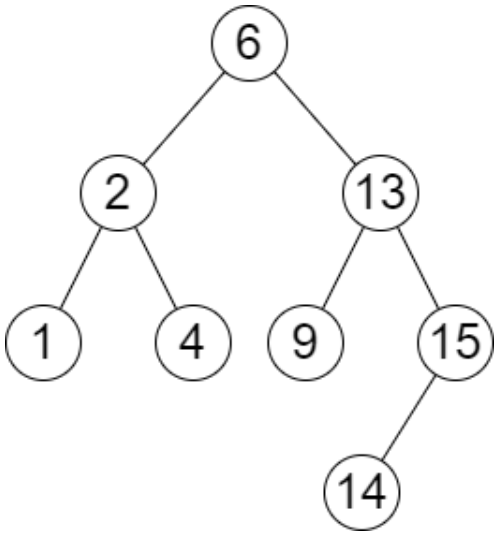
235.二叉搜索树的最近公共祖先：和普通二叉树的 LCA（236）对比着写。BST 的 LCA 不需要回溯，从根往下走，第一个落在 p 和 q 之间的节点就是答案。递归和迭代都极其简洁，是“BST 性质让问题降维”的典范。

3.5 搜索中的剪枝与记忆化

lc-2476 二叉搜索树最近节点查询：给你一个二叉搜索树的根节点 root，和一个由正整数组成、长度为 n 的数组 queries。请你找出一个长度为 n 的二维答案数组 answer，其中 answer[i]=[mini,maxi]：

mini 是树中≤queries[i]的最大值。如果不存在这样的值则使用-1 代替。

maxi 是树中≥queries[i]的最小值。如果不存在这样的值则使用-1 代替。



输入：root = [6,2,13,1,4,9,15,null,null,null,null,null,null,14],
queries = [2,5,16]
输出：[[2,2],[4,6],[15,-1]]

这个题的最好的思路是，利用中序遍历二叉搜索树得到递增数组，再对数组进行二分查找。使用二分查找可以避免直接对树查找，树有可能退化为链表导致时间复杂度增加，思路一的时间复杂度为：$O(n + q \cdot \log n)$，思路二的时间复杂度是：$O(q \cdot h)$，h 是树高，但是树退化成链表的时候会变成 $O(q \cdot n)$，因此用一。

算法实现如下：

```c
void inorder(struct TreeNode *root, int *result, int *n)
{
    if (root == NULL) return;
    inorder(root->left, result, n);
    result[(*n)++] = root->val;
    inorder(root->right, result, n);
}
```

先写中序遍历函数，遍历结果存放在数组 result 中。

下来写二分查找方法：

```c
int *midsearch(int *result, int n, int number)
{
    int *resulti = malloc(2 * sizeof(int));
    int low = 0, high = n - 1;
    if (number < result[low])
    {
        resulti[0] = -1;
        resulti[1] = result[low];
        return resulti;
    }
    if (number > result[high])
    {
        resulti[0] = result[high];
        resulti[1] = -1;
        return resulti;
    }
    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        if (result[mid] == number)
        {
            resulti[0] = resulti[1] = number;
            return resulti;
        }
        else if (result[mid] < number) low = mid + 1;
        else high = mid - 1;
    }
    resulti[0] = result[high];
    resulti[1] = result[low];
    return resulti;
}
```

这个二分查找函数很有意思。由于题目特殊的要求，需要分各种情况来讨论，每进行一次二分查找都要产出一个二元数组。先写上前两个标红的边界处理条件，下来再进行正常的二分查找，最后如果数组没有指定查找的元素，**而且此时 low 刚好大于 high 一个单位，根据题目要求，填写最后两个标红的语句即可**。

```
int** closestNodes(struct TreeNode* root, int* queries, int queriesSize,int*
returnSize, int**returnColumnSizes)
{
    int *sorted = malloc(100000 * sizeof(int));
    int n = 0;
    inorder(root, sorted, &n);
    int **ans = malloc(queriesSize * sizeof(int *));
    *returnColumnSizes = malloc(queriesSize * sizeof(int));
    for (int j = 0; j < queriesSize; j++)
    {
        ans[j] = midsearch(sorted, n, queries[j]);
        (*returnColumnSizes)[j] = 2;
    }
    *returnSize = queriesSize;
    return ans;
}
```

最后主函数调用前两个函数即可，顺便填一个题目要求的 returnColumnSizes。

这个题也是剪枝的代表，二分查找是经典的**剪枝**思想，一次砍去一半不合格的区间，大大减少时间复杂度和工作量。

lc-124 **最大路径和**，是动态规划的经典题目，有记忆化的体现（记住孩子的值），后面还有类似 DP 问题比如打家劫舍等等，后面会讲解。

三、树的结构与性质

这一部分主要是对前面所学知识的回顾与复习，大多内容已经提及，这一章本质上不是新的算法，而是对前两章（遍历、搜索）的重新分类和概念补漏。所以这一章的例题大多是已经做过的题，只是换了一个“看问题的角度”。

1. 树的基本概念（深度、高度、度、叶子、平衡因子）

lc-104 **二叉树的最大深度**：高度/深度概念的载体（DFS 章已写）

lc-111 **二叉树的最小深度**：注意“叶子节点”定义，只有左右都空才是叶子

前面都已经解决，下来关注完全二叉树：

lc-222 **完全二叉树的节点个数**：给你一棵完全二叉树的根节点 root，求出该树的节点个数。

此题虽然可以使用递归何层序遍历来求结点个数，但是要写出一个更高效的算法

所以最后采用二分查找+位运算的算法实现：

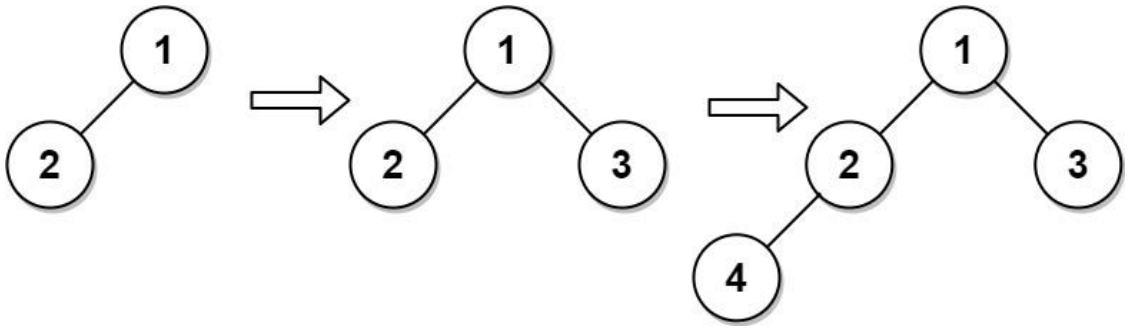
```
long long countNodes(struct TreeNode* root)
{
    long long hl=0,rl=0;
    for(struct TreeNode* left=root->left;left!=NULL;left=left->left) hl++;
    for(struct TreeNode* right=root->right;right!=NULL;right=right->right)rl++;
    if(hl==rl) return (1LL<<(hl+1))-1;
    return 1+countNodes(root->left)+countNodes(root->right);
}
```

对于一个满二叉树，左右子树的高度是一样的，直接带入公式求解；当左右子树高度不一致是，对左右子树各自进行递归，最后总可以求出结果。

这个算法的时间复杂度可以达到 $O(\log^2 n)$，比常规的 O(n)更快。

2. 特殊二叉树

lc-919 完全二叉树插入器：完全二叉树是每一层（除最后一层外）都是完全填充（即节点数达到最大）的，并且所有的节点都尽可能地集中在左侧。设计一种算法，将一个新节点插入到一棵完全二叉树中，并在插入后保持其完整。



输入：["CBTInserter", "insert", "insert", "get_root"], [[[1, 2]], [3], [4], []]

输出：[null, 1, 2, [1, 2, 3, 4]]

此题的核心思想是，利用层序遍历的方式将原版树的所有结点读取到一个队列上面，这个队列看做是数组，且我们人为的让这个**数组从 1 开始计数**。之后新插入的结点放在队列的最后面，同时新结点的下标也有了，**将新结点的下标除以二并下取整得到的就是父节点的下标**，题目要求插入后返回父节点，而且标号已知，直接返回即可。

这是二叉树的顺序存储方式，所以**偶数标号是左孩子，奇数是右孩子**。

算法思想很简单，但是作为 leetcode 算法设计题，总会有点猎奇成分在里面：

要求写一个类，里面包含树的创建，插入，读取 root，释放空间的函数，C 语言没有面向对象的能力，所以函数分开写，先写结构体定义如下：

```
#define CAP (1000+10000+5)
typedef struct
{
    struct TreeNode* root;
    struct TreeNode* Queue[CAP];
    int front,rear;
} CBTInserter;
```

定位：**四个函数跨调用共享的全部状态**。obj 在多次调用间传递状态，等价于 C++ 里 class 的成员变量（obj = this）。

```
class CBTInserter {
    TreeNode* root;  std::queue<TreeNode*> q;
public:
    CBTInserter(TreeNode* root);
    int insert(int val);
    TreeNode* get_root();
};
```

可以和 cpp 的写法做一个对比。

```
CBTInserter* cBTInserterCreate(struct TreeNode* root)
{
    CBTInserter *o = malloc(sizeof(CBTInserter));
    o->root=root;
    o->front=o->rear=0;
    o->Queue[++o->rear]=root;
    while(o->front<o->rear)
    {
        struct TreeNode* node=o->Queue[++o->front];
        if(node->left) o->Queue[++o->rear]=node->left;
        if(node->right) o->Queue[++o->rear]=node->right;
    }
    return o;
}
int cBTInserterInsert(CBTInserter* obj, int val)
{
    struct TreeNode *p = malloc(sizeof(struct TreeNode));
    p->val=val;
    p->left=p->right=NULL;
    int i=++obj->rear;
    struct TreeNode *parent=obj->Queue[i/2];
    if(i%2==0) parent->left=p;
    else parent->right=p;
    obj->Queue[i]=p;
    return parent->val;
}
struct TreeNode* cBTInserterGet_root(CBTInserter* obj)
{
    return obj->root;
}
void cBTInserterFree(CBTInserter* obj)
{
    free(obj);
}
```

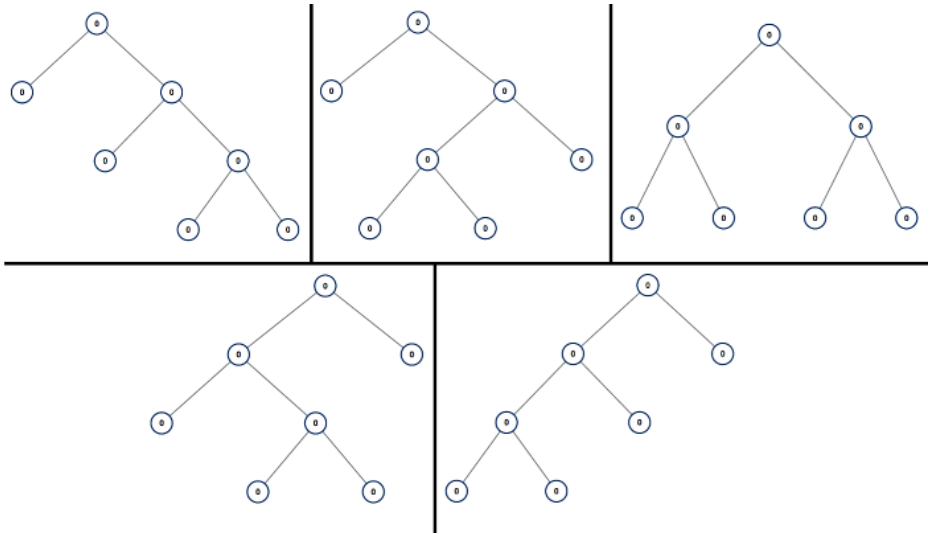
中间的算法函数 cBTInserterInsert 才是此题的核心，与之前的思想一致。

lc-894 **所有可能的真二叉树**：给你一个整数 n，找出所有可能含 n 个节点的真二叉树并以列表形式返回。答案中每棵树的每个节点都必须符合 Node.val==0。**答案的每个元素都是一棵真二叉树的根节点**。可按任意顺序返回最终的真二叉树列表。

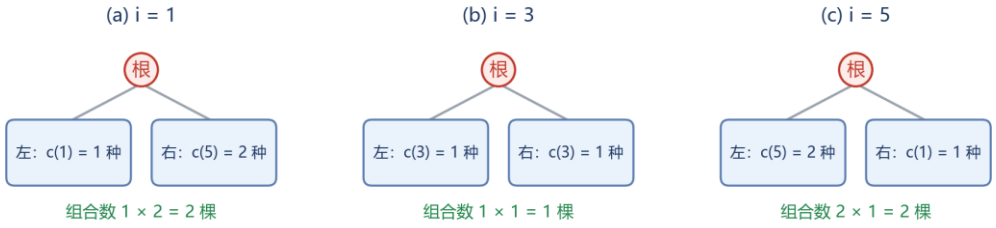
真二叉树是一类二叉树，**树中每个节点恰好有 0 或 2 个子节点**。例如：

输入：n = 7

输出：
[[0, 0, 0, null, null, 0, 0, null, null, 0, 0], [0, 0, 0, null, null, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, null, null, null, null, 0, 0], [0, 0, 0, 0, 0, null, null, 0, 0]]



对于 7 个结点的真二叉树，可构造如上 5 种类型的树，是枚举的思想。



c(7) = c(1)·c(5) + c(3)·c(3) + c(5)·c(1) = 2 + 1 + 2 = 5

对于一个真二叉树，节点数必须是奇数，如果传入的参数 n 是一个偶数，说明不符合要求。而且对于符合要求的奇数 n，根节点算一个结点，假如说**左子树有 i 个结点，那右子树只能是 n-1-i 个结点**，同时每一个 i 也必须是奇数，因为树是递归定义的，所以右子树的 n-1-i 亦是奇数。

然后对 i=1，3，5，7……往后遍历，观察左右子树在 i 情况下的真二叉树个数，之后左子树的个数乘以右子树的个数，就是在 i 情况下的一对组合，最后对 i 求和即可得到全部的真二叉树数目，后面开辟数组时需要。

```
int count(int n)
{
    int total=0;
    if(n==1) return 1;
    for(int i=1;i<n-1;i+=2)
    {
        total+=count(i)*count(n-i-1);
    }
    return total;
}
```

求真二叉树个数的函数。（i 只能是奇数）

```
struct TreeNode* makenewnode(int val)
{
    struct TreeNode* root=(struct TreeNode*)malloc(sizeof(struct TreeNode));
    root->val=val;
    root->left=root->right=NULL;
    return root;
}
```

开辟一个新节点，题目要求每个结点的值 val 都为 0

```
struct TreeNode** build(int n,int *returnSize)
{
    if(n==1)
    {
        struct TreeNode** res=malloc(sizeof(struct TreeNode));
        res[0]=makenewnode(0);
        *returnSize=1;
        return res;
    }
    struct TreeNode** res=malloc(count(n)*sizeof(struct TreeNode));
    int k=0;
    for(int i=1;i<n-1;i+=2)
    {
        int lh,rh;
        struct TreeNode** left=build(i,&lh);
        struct TreeNode** right=build(n-i-1,&rh);
        for(int a=0;a<lh;a++)
        {
            for(int b=0;b<rh;b++)
            {
                struct TreeNode* root=makenewnode(0);
                root->left=left[a];
                root->right=right[b];
                res[k++]=root;
            }
        }
        free(left);free(right);
    }
    *returnSize=k;
    return res;
}
```

标红的三个 for 循环就是前面提到的枚举，同时内部用递归的方式求出左右子树的列表往上传递，这个题是**枚举+递归+分治**的综合题。

数组要一次 malloc 恰好大小，于是写了个纯数数版 countFBT——和建树共用同一条递推 $c(n) = \Sigma\ c(i) \cdot c(n-1-i)$（i 取奇数）。它顺手成了建树版的自检对照。实测棵数就是 Catala 数。

最后主函数调用：

```
struct TreeNode** allPossibleFBT(int n, int* returnSize)
{
    if(n%2==0)
    {
        *returnSize=0;
        return NULL;
    }
    return build(n,returnSize);
}
```

对不同的 n 进行测试，看看各自有多少种真二叉树：

n	真二叉树棵数 c(n)
1	1
3	1
5	2
7	5
9	14
11	42
13	132
15	429
17	1430
19	4862

n≤20 时上限 4862 棵，直接递归瞬间跑完，不需要记忆化。

lc-1382 **将二叉搜索树变平衡**：给你一棵二叉搜索树，请你返回一棵平衡后的二叉搜索树，新生成的树应该与原来的树有着相同的节点值。如果有多种构造方法，请你返回任意一种。

先把 BST 中序遍历一次得到一个有序数组，再将这个有序数组构造成一个 BST，这个 BST 是一定平衡的 AVL，具体操作和 lc-108 一样，不再赘述。

3. 堆

堆是一种**物理上用数组顺序存储，逻辑上是一种完全二叉树的**数据结构。堆分为两类：大根堆和小根堆，大根堆是根结点的值大于左右孩子结点的值，小根堆是根结点的值都小于左右孩子结点的值，同时其子树保持这个性质。

堆最常见的应用就是**堆排序**，如果想获得递增序列就构造小根堆，反之就构造大根堆，堆排序是不断读取堆顶元素，之后再重新调整堆使之继续满足大/小根堆的过程，其中最关键的函数就是 heapPush 和 heapPop。

heapPush 是入堆操作，入堆总是从堆尾进入，也就是完全二叉树的最后一层的最后一个结点后插入，之后调整堆，以小根堆为例，调整就是让**小结点上浮**的过程。

heapPop 是出堆操作，出堆总是从堆头出去，也就是树的根结点，出去之后用堆尾的元素占位，下来又要调整堆，还是以小根堆为例，此时要做的就是让**大结点下沉**，同时下沉的过程需要比较左右孩子结点值的大小，选择左右孩子结点值较小的一个与根结点的值交换，多一次比较。

堆的高效性体现在：树的高度 O(log n)，上浮/下沉最多走一条路径。

堆适合干什么？ **答案：动态地、反复地取当前极值**。

典型场景：

1）优先队列（Dijkstra、任务调度）

2）Top K（前 K 大用小根堆，前 K 小用大根堆）

3）动态中位数（对顶堆）

4）合并 K 个有序序列

5）贪心 + 反悔（加油、课程表 III）

堆的入堆和出堆函数如下：

```
void heapPush(int *heap,int *size,int x)
{
    int j=(*size)++;
    heap[j]=x;
    while(j>0)
    {
        int p=(j-1)/2;
        if(heap[j]>=heap[p]) break;
        int tmp=heap[p];heap[p]=heap[j];heap[j]=tmp;
        j=p;
    }
}
```

入堆在堆尾，要和它的父亲比较大小，因为物理存储是数组形式，所以可以直接求出父结点的下标来锁定父结点，如果发现不满足小根堆就交换，大根堆则相反。

```
void heapPop(int *heap,int *size)
{
    (*size)--;
    heap[0]=heap[(*size)];
    int p=0;
    while(1)
    {
        int left=2*p+1,right=2*p+2,m=p;
        if(left<(*size) && heap[left]<heap[m]) m=left;
        if(right<(*size) && heap[right]<heap[m]) m=right;
        if(p==m) break;
        int tmp=heap[p];heap[p]=heap[m];heap[m]=tmp;
        p=m;
    }
}
```

这是出堆函数，堆顶元素换上去后，多一次两个孩子之间的比较，这里的书写采用打擂台的方式找出最小值，直到符合小根堆的条件结束，大根堆则相反。

这两个函数都是 void 类型，根据之后要解决的问题不同，函数类型也要改变，比如 lc-23 拼接链表的时候就需要返回结点类型，改成 struct ListNode*型。

lc-23 合并 K 个升序链表：给你一个链表数组，每个链表都已经按升序排列。请你将所有链表合并到一个升序链表中，返回合并后的链表。例如：

输入：lists = [[1,4,5],[1,3,4],[2,6]]
输出：[1,1,2,3,4,4,5,6]

lc-23 题的具体过程如图所示：



算法实现：

```
struct ListNode* mergeKLists(struct ListNode** lists, int listsSize)
{
    if(lists==NULL || listsSize==0) return NULL;
    struct ListNode* dummy=(struct ListNode*)malloc(sizeof(struct ListNode));
    int size=0;
    struct ListNode* tail=dummy;
    dummy->next=NULL;
    struct ListNode** heap=(struct ListNode**)malloc(listsSize*sizeof(struct
ListNode* ));
    for(int i=0;i<listsSize;i++)
    {
        if(lists[i]!=NULL) heapPush(heap,&size,lists[i]);
    }
    while(size>0)
    {
        struct ListNode* node=heapPop(heap,&size);
        tail->next=node;
        tail=node;
        if(node->next!=NULL) heapPush(heap,&size,node->next);
```

```
    }
    tail->next=NULL;
    free(heap);
    return dummy->next;
}
```

注意标红的地方不可以写成循环 K 次，判断条件必须是堆是否为空，因为初始化的堆是 K 的子链表的第一个元素所构成的，在经历出堆操作之后，需要将其余的元素入堆，如此循环才能使得所有子链表的元素被使用。

lc-215 **数组中的第 K 个最大元素**：给定整数数组 nums 和整数 k，请返回数组中第 k 个最大的元素。例如：

输入：[3,2,1,5,6,4], k = 2

输出：5

依旧使用堆的操作：

```
int findKthLargest(int* nums, int numsSize, int k)
{
    int *heap=malloc((k+1)*sizeof(int ));
    int size=0;
    for(int i=0;i<numsSize;i++)
    {
        heapPush(heap,&size,nums[i]);
        if(size>k) heapPop(heap,&size);
    }
    int ans=heap[0];
    free(heap);
    return ans;
}
```

lc-295 **数据流的中位数**：中位数是有序整数列表中的中间值。如果列表的大小是偶数，则没有中间值，中位数是两个中间值的平均值。

实现 MedianFinder 类:

1）MedianFinder() 初始化 MedianFinder 对象。

2）void addNum(int num) 将数据流中的整数 num 添加到数据结构中。

3）double findMedian() 返回到目前为止所有元素的中位数。

例如：

输入：

["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]

[[], [1], [2], [], [3], []]

输出：

[null, null, null, 1.5, null, 2.0]

这个题和 lc-919 完全二叉树生成器很像，都要写类来实现，但是 C 语言没有类，实现起来会略微复杂一些，而且这个题实现起来的细节比 919 更多。

首先要清楚**数据流**是什么，数据流不是一个固定的数组，而是会时不时添加元素进来，在这个动态的过程中找到中位数，对于这种动态的题目就要使用堆。

在这里使用堆是高效的做法，核心算法是：将于数组的前半部分构建成一个大根堆，后半部分构建成一个小根堆，大根堆的结点数≥小根堆结点数（大于的时候也只是大 1），**最后取大根堆的堆顶元素，就是数据流的中位数，如果数据流长度是偶数，那就返回大根堆和小根堆堆顶元素的平均数**（double 型）。

```
typedef struct
{
    int *heapmax;
    int size1;
    int *heapmin;
    int size2;
} MedianFinder;

MedianFinder* medianFinderCreate()
{
    MedianFinder *obj=malloc(sizeof(MedianFinder));
    obj->heapmax=malloc(MAXSIZE *sizeof(int ));
    obj->heapmin=malloc(MAXSIZE *sizeof(int ));
    obj->size1=0;
    obj->size2=0;
    return obj;
}

void medianFinderAddNum(MedianFinder* obj, int num)
{
    maxPush(obj->heapmax,&obj->size1,num);
    minPush(obj->heapmin,&obj->size2,maxPop(obj->heapmax,&obj->size1));
    if(obj->size1<obj->size2)
    {
        maxPush(obj->heapmax,&obj->size1,minPop(obj->heapmin,&obj->size2));
    }
}
```

严格按照 leetcode 给的函数接口来写，第一步先初始化两个堆，下来就是构建这两个堆，这里的构建堆采用了一个非常巧妙地方法，对于已有的两个堆，**新来一个数据元素，先让 num 无脑进入左边的大根堆，调整好之后把大根堆的堆顶扔给小根堆，如果这时小根堆结点数大于了大根堆，就需要给大根堆匀一个元素过去**。

实际上，这一系列操作正是将一个无序的数据流变得有序起来，左边为什么要给右边堆顶元素？就是把左边最大的往右边走，右边最小的同时又往左边走。

最后构建好的两个根堆直接就可以读取到中位数了，构建堆的过程和数据流元素的有序性无关，不论什么顺序输入，最后在堆的领域里都会变成局部有序，所以大可不必担心输入顺序的问题！

```c
double medianFinderFindMedian(MedianFinder* obj)
{
    if(obj->size1>obj->size2) return obj->heapmax[0];
    return (obj->heapmax[0]+obj->heapmin[0])/2.0;
}

void medianFinderFree(MedianFinder* obj)
{
    free(obj->heapmin);
    free(obj->heapmax);
    free(obj);
}
```

这两个成员函数是读取中位数和释放空间，前面还有四个函数省略了，分别是 maxPush，maxPop，minPush，minPop，是大小根堆的入堆和出堆函数，C 语言实现起来就要比 C++麻烦一些，这个题用 C 语言写了 102 行代码。

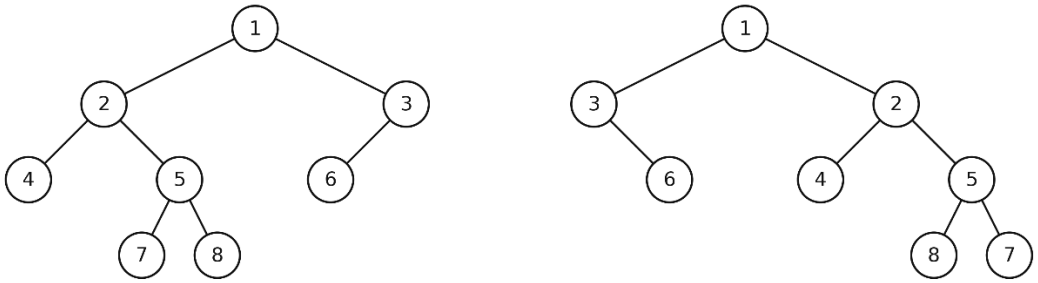
4. 树的对称与镜像

lc-101 **对称二叉树**：给一个二叉树的根节点 root 检查它是否轴对称。（已写）
lc-226 **翻转二叉树**：给一棵二叉树的根节点 root，翻转这棵二叉树。（已写）

lc-951 **翻转等价二叉树**：我们可以为二叉树 T 定义一个翻转操作，即选择任意节点，然后交换它的左子树和右子树。只要经过一定次数的翻转操作后，能使 X 等于 Y，我们就称二叉树 X 翻转等价于二叉树 Y。

这些树由根节点 root1 和 root2 给出。如果两个二叉树是翻转等价的树，则返回 true，否则返回 false。例如：

翻转等价二叉树 · LeetCode 951



翻转节点 1、3、5 后，两棵树完全相同

输入：root1 = [1,2,3,4,5,6,null,null,null,7,8]，root2 = [1,3,2,null,6,4,5,null,null,null,null,8,7]
输出：true
解释：我们翻转值为 1，3 以及 5 的三个节点。

采用递归方式判断：

```
bool flipEquiv(struct TreeNode* root1, struct TreeNode* root2)
{
    if(root1==NULL && root2==NULL) return true;
    if(root1==NULL || root2==NULL) return false;
    if(root1->val!=root2->val) return false;
    bool notflip=flipEquiv(root1->left,root2->left) &&
flipEquiv(root1->right,root2->right);
    bool flip=flipEquiv(root1->left,root2->right) &&
flipEquiv(root1->right,root2->left);
    return notflip || flip;
}
```

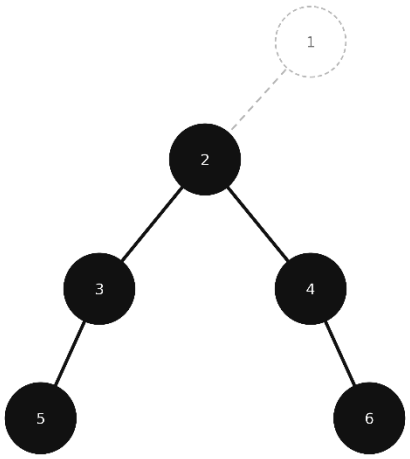
值得注意的是这个题目要求你判断翻转或者不翻转是否一样，返回的是一个 bool 型变量，所以分两类，第一类不翻转就是左对左右对右，第二类翻转就是左对右右对左交叉一下，**并不是真的叫你把树翻转，这样会很麻烦，只用判断就可以了**。

5. 树的子树与同构

lc-572 **另一棵树的子树**：嵌套 DFS（DFS 章已写）
lc-100 **相同的树**：同构判断的基础（DFS 章已写）

6. 树的直径、最大路径和

lc-543 **二叉树的直径**：给你一棵二叉树的根节点，返回该树的直径。二叉树直径是指树中任意两个节点之间最长路径的长度。这条路径可能经过也可能不经过根节点 root。**两节点之间路径的长度由它们之间边数表示**。例如：



这个例子说明了，最长的半径可以不经过 root 结点，路径为：5->3->2->4->6，直径是经过的边数，也就是 4。

这个题的关键是后序遍历，先递归到叶结点再一层一层往上退层，在向上走的过程中记录下当前结点作为**拐点**的最大直径也就是 trough 值，和公共的 best 值作比较，如果 through 更大就更新 best 记录下来。

同时要往上 return，给父结点传递的是孩子结点最长的子路径，而不是都要，因为**父结点的角色同样也是拐点**，这就决定了只能选择一条路径。

这样，左子树递归完了接着就是右子树递归，最后得到的 left 和 right 值进行比较即可，主函数调用的时候输出 best 即可，这就是我们要的直径结果。

注意，我们求直径实际上是求任意两个结点之间的边数之和。我们把直径初始化为-1，对于当前的拐点 u 来说直径是左边数+右边数，左右边数分别是：left+1 和 right+1，**合起来就是 left+right+2**;

算法实现如下：

```
int height(struct TreeNode* u,int *best)
{
    if(u==NULL) return -1;
    int left=height(u->left,best);
    int right=height(u->right,best);
    int through=left+right+2;
    if(through>*best) *best=through;
    return (left>right?left:right)+1;
}
int diameterOfBinaryTree(struct TreeNode* root)
{
    int best=0;
    height(root,&best);
    return best;
}
```

标红的三句话是重点，这个题相当于**是 lc-124 最大路径和的前菜题**，在 DFS 部分已经写了 124 题，用的和这个题一样的 dfs 后序遍历方法+往上传值。

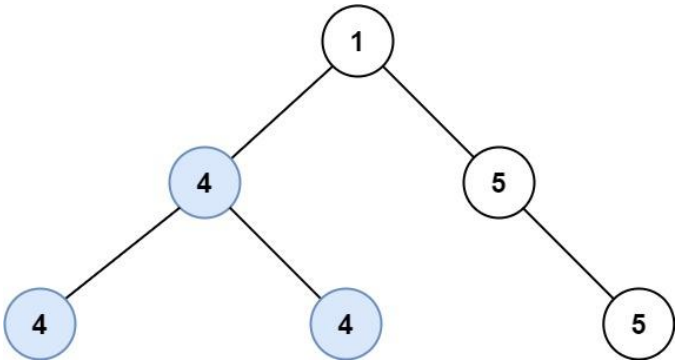
lc-124 二叉树中的最大路径和：543 的加权进阶版。

再版 124 的代码放出来，和 543 做对比：

```
int postdfs(struct TreeNode*root)
{
    if(root==NULL) return 0;
    int left=max(postdfs(root->left),0);
    int right=max(postdfs(root->right),0);
    maxSum=max(maxSum,root->val+right+left);
    return root->val+max(left,right);
}
```

重点对照标红的区域！

lc-687 最长同值路径：给定一个二叉树的 root，返回最长的路径的长度，这个路径中的每个节点具有相同值。这条路径可以经过也可以不经过根节点。例如：



输入：root = [1,4,5,4,4,5]

输出：2

在我看来这个题算是 543 的孪生兄弟题，无非就是比 543 多了一个判断左右孩子和自己是不是相等的 if 语句（因为 u 自己是拐点，路径上的元素必须相同）。

算法实现如下：

```
int postdfs(struct TreeNode *u, int *best)
{
    if(u==NULL) return -1;
    int left=postdfs(u->left,best);
    int right=postdfs(u->right,best);
    if (u->left==NULL || u->left->val!=u->val) left=-1;
    if (u->right==NULL || u->right->val!=u->val) right=-1;
    int through =left+right+2;
    if (through>*best) *best=through;
    return (left>right?left:right)+1;
}
```

多写的这两个 if 看一下，也就是说，如果发现 u 的左边或者右边和自己不一样，**直接将他们暴力清空为-1，即不符合要求的部分直接砍掉**。

稍微总结一下，这三个题目其实都算是 DP 动态规划问题，由于动态规划需要知道后面结点的信息，因此常常用到后序遍历的方式来解决。对于上面的三个问题，可以写出他们各自的目标函数和约束条件，先展示总体：

$$f(u) = Single\big(u, f(left), f(right)\big)$$

$$through(u) = Through\big(u, f(left), f(right)\big)$$

$$G = maxthrough(u)$$

	$f(NULL)$	单侧h	$Single: f(u)$	$Through: through(u)$
lc-543	-1	无	max(f(l),f(r))+1	f(l)+f(r)+2
lc-124	0	无	val+max(f(l),f(r))	val+max(f(l),0)+max(f(r),0)
lc-687	-1	h_x	max(hl,hr)+1	hl+hr+2

上表展示了这三个问题各自的函数是什么，我们建立的模型和代码求解是一样的。其中h_x表示为：

$$h_x = \begin{cases} f(x), ok \\ -1, \rightarrow ok \end{cases}$$

意味着不符合要求的支路需要被砍掉。

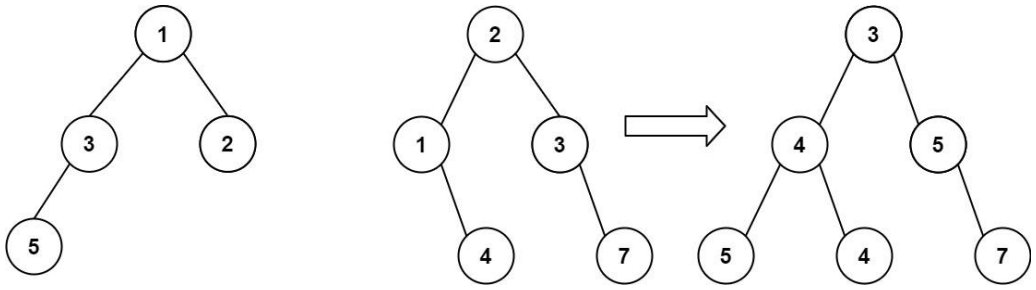
$f(u)$是以 u 为起点，只往一条腿延伸的最优值，其值向上传递给父亲；$through(u)$是以 u 为拐点的最优路径值，就地结算并更新全局 best 值；G（即 best）是全树所有 through 的最大值，也是我们写算法的目的。

解决动态规划问题的一般步骤如上。那是不是一看到后序遍历就一定是动态规划问题呢？不一定，后序遍历只是一个工具，单纯的后序遍历也只是得到一个遍历序列。只有当看见题目要**求包含向上传递某某某**这个信息的时候，才要往动态规划方向去思考。

就以这三个题为例，我们需要思考这几点再判断是不是 DP 问题：
1）子返回的东西，是不是父节点计算所需要的“子问题解”？
2）父节点的解，能不能由孩子的解“合并/转移”得到？
3）是否需要在节点处结算一个“全局最优”，而单边返回值只是原料？
如果三个都是，那基本就是树形 DP 问题了。

7. 二叉树的合并

lc-617 合并两棵二叉树：给你两棵二叉树：root1 和 root2。
想象一下，当你将其中一棵覆盖到另一棵之上时，两棵树上的一些节点将会重叠（而另一些不会）。你需要将这两棵树合并成一棵新二叉树。合并的规则是：如果两个节点重叠，那么将这两个节点的值相加作为合并后节点的新值；否则，不为 null 的节点将直接作为新二叉树的节点。
注意：合并过程必须从两个树的根节点开始。例如：



输入：root1 = [1,3,2,5], root2 = [2,1,3,null,4,null,7]
输出：[3,4,5,5,4,null,7]

算法实现如下：

```
struct TreeNode* mergeTrees(struct TreeNode* root1, struct TreeNode* root2)
{
    if(root1==NULL) return root2;
    if(root2==NULL) return root1;
    struct TreeNode* node=malloc(sizeof(struct TreeNode));
    node->val=root1->val+root2->val;
    node->left=mergeTrees(root1->left,root2->left);
    node->right=mergeTrees(root1->right,root2->right);
    return node;
}
```

四、树上的动态规划（树形 DP）

1. 自底向上的信息传递

lc-543 二叉树的直径：刚刚做过的最经典的自底向上传递的问题，同样的 124，687 都是这个思想，不再赘述。

2. 自顶向下的信息传递

lc-1372 二叉树中的最长交错路径：给一棵以 root 为根的二叉树，二叉树中的交错路径定义如下：

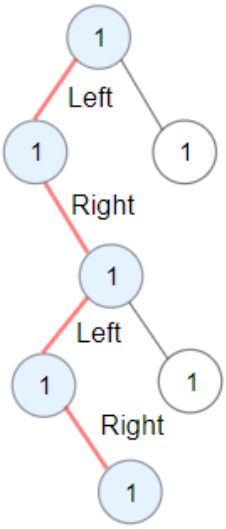
1）选择二叉树中任意节点和一个方向（左或者右）；

2）若前进方向为右，则移动到当前结点的右结点，否则移动到它的左结点；

3）改变前进方向：左变右或者右变左；

4）重复第二步和第三步，直到你在树中无法继续移动；

5）交错路径的长度定义为：访问过的节点数目-1。例如：



输入：root = [1,1,1,null,1,null,null,1,1,null,1]

输出：4

解释：蓝色节点为树中最长交错路径（左 -> 右 -> 左 -> 右）。

注意：最后的结果一定是一个锯齿状的路径，所以在写算法的时候一定要注意对各种情况的考虑！

算法实现如下：

```
int maxLen;
void dfs(struct TreeNode* node, int isLeft, int len)
{
    if(node==NULL) return;
    if(len>maxLen) maxLen=len;
    if(isLeft)
    {
        dfs(node->right,0,len+1);
        dfs(node->left,1,1);
    }
    else
    {
        dfs(node->left,1,len+1);
        dfs(node->right,0,1);
    }
}
```

```c
int longestZigZag(struct TreeNode* root)
{
    maxLen=0;
    if (root==NULL) return 0;
    dfs(root->left,1,1);
    dfs(root->right,0,1);
    return maxLen;
}
```

分成两部分讨论，也就是刚向左走和刚向右走完的情况。以第一种为例，刚刚向左走完就只能向右走，或者重开一个路径，为什么会有重开？**因为如果你遇到了连续的三个结点，类似链表的样式**，如果继续往右走，结果就不符合锯齿状的条件了，因此必须重开一个路，**重开之后，路径长度就定为 1**，因为之前的都丢失了。

注意：isLeft 的值代表刚刚干了什么，1 代表刚走完左边，0 代表右边。

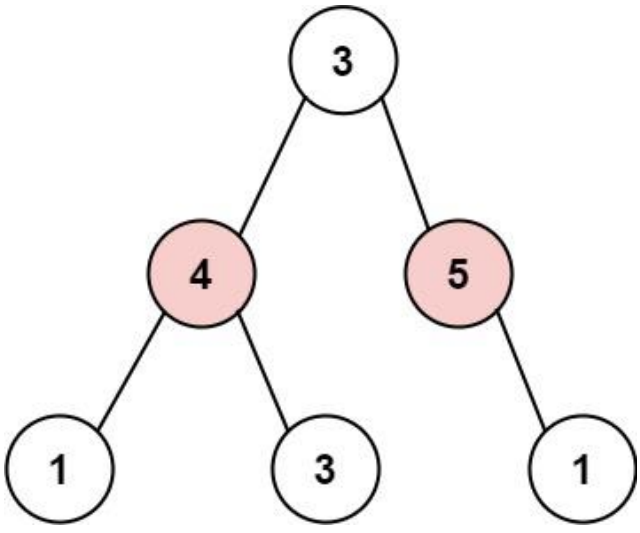
这个题属于是广义的动态规划问题，因为也牵扯到了状态的转移，只不过搜寻方式是先序而不是后序，上一个问题是孩子向父亲传递，这个问题是父亲向孩子传递，同时增设一个标志 isLeft 记录之前的状态。

3. 打家劫舍问题

这里只写打家劫舍 III，因为 198/213 是线性 DP，约束相同但数据在数组上。337 把同样的约束搬到二叉树上，核心变化是：**决策依赖子树返回的状态，必须后序遍历**，树形 DP 的入口题。

lc-337 打家劫舍 III：小偷又发现了一个新的可行窃的地区。这个地区只有一个入口，我们称之为 root 。除了 root 之外，每栋房子有且只有一个“父“房子与之相连。一番侦察之后，聪明的小偷意识到“这个地方的所有房屋的排列类似于一棵二叉树”。如果两个直接相连的房子在同一天晚上被打劫，房屋将自动报警。

给定二叉树的 root。返回在不触动警报的情况下，小偷能够盗取的最高金额。



输入：root = [3,4,5,1,3,null,1]
输出：9
解释： 小偷一晚能够盗取的最高金额 4 + 5 = 9

我们要知道，相邻的两个点是不能同时被抢的，所以分两个情况讨论：

1）父结点被抢了，孩子不能抢；
2）父结点没被抢，两个孩子都能抢。

1. 建立数学模型如下：

给定二叉树T，结点u有权值$w(u)$，选一个节点集合S，满足：

$$\forall u \in S,\ u\text{的父节点和子节点都不在}S$$

目标是：

$$\max \sum (u)$$

2. 求解数学模型：状态转移是以下关系式，用这个写代码实现：

1）选了u，则u的左右孩子L,R都不能选，只能取它们的$skip$：

$$take(u) = \omega(u) + skip(Left) + skip(Right)$$

2）不选u，则孩子可以选也可以不选，每个孩子取两种状态的较大值：

$$skip(u) = max\big(take(Left), skip(Left)\big) + max\big(take(Right), skip(Right)\big)$$

每一个算法都要处理边界情况：$take(\phi) = 0, skip(\phi) = 0$。

最后程序要返回的结果就是$max\big(take(root), skip(root)\big)$.

算法实现如下：

```
struct Rob
{
    int take;
    int skip;
};
struct Rob dfs(struct TreeNode *u)
{
    if (u==NULL) return (struct Rob){0,0};
    struct Rob L=dfs(u->left);
    struct Rob R=dfs(u->right);
    struct Rob cur;
    cur.take=u->val+L.skip+R.skip;
    cur.skip=max(L.take,L.skip)+max(R.take,R.skip);
    return cur;
}
int rob(struct TreeNode *root)
{
    struct Rob r=dfs(root);
    return max(r.take,r.skip);
}
```

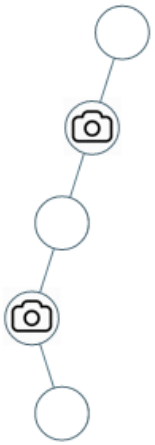
返回值要恰好承载父节点决策所需的信息，多传没用，少传不够。

4. 二叉树种最大路径和

lc-124 **二叉树种最大路径和**，这个题已经提及了很多次，不再叙述。

5. 监控二叉树/状态机 DP

lc-968 **监控二叉树**：给定一个二叉树，我们在树的结点上安装摄像头。结点上的**每个摄影头都可以监视其父对象、自身及其直接子对象**。计算监控树的所有结点所需的最小摄像头数量。例如：



输入：[0,0,null,0,null,0,null,null,0]
输出：2
解释：需要至少两个摄像头来监视树的所有节点。

1.建立数学模型：

给定一棵二叉树。可以在任意结点上安装摄像头，一个摄像头能覆盖它自己、它的父亲、它的左右孩子。要选择尽量少的结点安装摄像头，使树中每个点被覆盖。

把树记为$T=(V,E)$，根为r。对节点v，记 $p(v)$为父亲，$L(v),R(v)$ 为左右孩子。定义v的闭邻域：

$$N[v]=\{v\}\cup\{p(v)\}\cup\{L(v)\}\cup\{R(v)\}$$

符号说明：

符号	含义
$T=(V,E)$	二叉树，V 为节点集，E 为边集
r	根节点
$p(v)$	v 的父亲；根没有父亲
$L(v),R(v)$	v 的左右孩子，可能为空
$N[v]$	闭邻域：v、父亲、左右孩子
x_v	0-1 变量：是否在 v 装摄像头
S	安装了摄像头的节点集合

使用 0/1 整数规划模型，建立符合题意的完整模型如下：

$$\min \sum_{v\in V} x_v$$

$$\text{s.t.} \sum_{u\in N[v]} x_u \geq 1,\ \forall v\in V$$

$$x_v\in\{0,1\},\ \forall v\in V$$

三个式子分别是目标函数，约束条件，以及决策变量。

整数规划给出“要什么”，下面的状态递归给出“怎么算”，即建模的求解。每个节点向父亲汇报且只汇报一个状态。

2. 求解数学模型：

状态	含义	对父亲的要求
NEED	自己没被覆盖，且自己没装摄像头	请父亲装一个
COVERED	自己已被覆盖，但自己没装	无
CAMERA	自己装了摄像头	无；还能覆盖父亲

空节点不需要被覆盖，也不提供摄像头，所以规定：

$$s(\phi) = COVERED$$

对真实节点 v，设 $a = L(v)$、$b = R(v)$。三个状态的转移为：

$$s(v) = CAMERA \ \ if \ \ s(a) = NEED \ or \ s(b) = NEED$$

$$s(v) = COVERED \ \ if \ no \ child \ is \ NEED \ and \ \ (s(a) = CAMERA \ or \ s(b) = CAMERA)$$

$$s(v) = NEED \ \ otherwise$$

最终摄像头集合与数量：

$$S = \{v \in V : s(v) = CAMERA\} \cup (\{r\} \ if \ s(r) = NEED \ else \ \emptyset)$$

$$K = |S|$$

所有的建模和求解都写完了，根据他们写出 C 语言代码：

```
enum
{
    NEED = 0,
    COVERED = 1,
    CAMERA = 2
};
int dfs(struct TreeNode *u, int *cameras)
{
    if (u==NULL) return COVERED;
    int l=dfs(u->left,cameras);
    int r=dfs(u->right,cameras);
    if (l==NEED || r==NEED)
    {
        (*cameras)++;
        return CAMERA;
    }
    if (l==CAMERA || r==CAMERA) {return COVERED;}
    return NEED;
}
int minCameraCover(struct TreeNode *root)
{
    int cameras=0;
    if (dfs(root,&cameras) == NEED)
    {
        cameras++;
    }
    return cameras;
}
```

转移的优先级不能乱：先看 NEED，再看 CAMERA，最后才是全 COVERED。若一个孩子是 NEED、另一个是 CAMERA，必须先满足 NEED 孩子，否则它会永远没人覆盖。

最后注意一点：**这三个状态是互斥的**，之后需要使用枚举类型封装起来，不能使用结构体。